Experiment 'p_4_1_a_sum_even_numbers' Results

June 3, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_4_1_a_sum_even_numbers
Natural language statement: Write a method with loops that computes the sum of all even numbers between 2 and 100 (inclusive).
Method signature: p_4_1_a_sum_even_numbers() returns (sum: int)

Ensures

- `sum == 2550`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method with loops that computes the sum of all even numbers between 2 and 100 (
    ↪ inclusive).

The signature should be:

p_4_1_a_sum_even_numbers() returns (sum: int)

The method should respect the following contract:

ensures sum == 2550

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DmdxCZJR5XJlmoi5R22EGFvbYA48J
Model: gpt-5.5-2026-04-23
Created at: 1780485698
Finish reason: stop
Usage: 268 tokens in, and 638 tokens out

```
{
  sum := 0;
  var n := 1;
  while n <= 50
    invariant 1 <= n <= 51
    invariant sum == n * (n - 1)
    decreases 51 - n
  {
    sum := sum + 2 * n;
    n := n + 1;
  }
  assert n == 51;
  assert sum == 51 * 50;
  assert sum == 2550;
}
```

Final Program

```
method p_4_1_a_sum_even_numbers() returns (sum: int)
       ensures sum == 2550
{
  sum := 0;
  var n := 1;
  while n <= 50
    invariant 1 <= n <= 51
    invariant sum == n * (n - 1)
    decreases 51 - n
  {
    sum := sum + 2 * n;
    n := n + 1;
  }
  assert n == 51;
  assert sum == 51 * 50;
  assert sum == 2550;
}
```

Total Token Usage

Input tokens: 268
Output tokens: 638
Reasoning tokens: 512
Sum of 'total tokens': 906

Experiment Timings

Overall Experiment started at 1780485697460, ended at 1780485715473, lasting 18013ms (18.01 seconds)
Iteration #1 started at 1780485697465, ended at 1780485715473, lasting 18008ms (18.01 seconds)

Experiment 'p_4_1_b_sum_squares' Results

June 3, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_4_1_b_sum_squares
Natural language statement: Write a method with loops that computes the sum of all squares between 1 and 100 (inclusive).
Method signature: p_4_1_b_sum_squares() returns (sum: int)

Ensures

- `sum == 338350`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method with loops that computes the sum of all squares between 1 and 100 (
    ↪ inclusive).

The signature should be:

p_4_1_b_sum_squares() returns (sum: int)

The method should respect the following contract:

ensures sum == 338350

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DmdxUVVszZRP2338lQ8GpnpFhRf3j
Model: gpt-5.5-2026-04-23
Created at: 1780485716
Finish reason: stop
Usage: 266 tokens in, and 6552 tokens out

```
{
  sum := 0;
  var i := 1;
  while i <= 100
    invariant 1 <= i <= 101
    invariant i == 1 ==> sum == 0
    invariant i == 2 ==> sum == 1
    invariant i == 3 ==> sum == 5
    invariant i == 4 ==> sum == 14
    invariant i == 5 ==> sum == 30
    invariant i == 6 ==> sum == 55
    invariant i == 7 ==> sum == 91
    invariant i == 8 ==> sum == 140
    invariant i == 9 ==> sum == 204
    invariant i == 10 ==> sum == 285
    invariant i == 11 ==> sum == 385
    invariant i == 12 ==> sum == 506
    invariant i == 13 ==> sum == 650
    invariant i == 14 ==> sum == 819
    invariant i == 15 ==> sum == 1015
```

```
invariant i == 16 ==> sum == 1240
invariant i == 17 ==> sum == 1496
invariant i == 18 ==> sum == 1785
invariant i == 19 ==> sum == 2109
invariant i == 20 ==> sum == 2470
invariant i == 21 ==> sum == 2870
invariant i == 22 ==> sum == 3311
invariant i == 23 ==> sum == 3795
invariant i == 24 ==> sum == 4324
invariant i == 25 ==> sum == 4900
invariant i == 26 ==> sum == 5525
invariant i == 27 ==> sum == 6201
invariant i == 28 ==> sum == 6930
invariant i == 29 ==> sum == 7714
invariant i == 30 ==> sum == 8555
invariant i == 31 ==> sum == 9455
invariant i == 32 ==> sum == 10416
invariant i == 33 ==> sum == 11440
invariant i == 34 ==> sum == 12529
invariant i == 35 ==> sum == 13685
invariant i == 36 ==> sum == 14910
invariant i == 37 ==> sum == 16206
invariant i == 38 ==> sum == 17575
invariant i == 39 ==> sum == 19019
invariant i == 40 ==> sum == 20540
invariant i == 41 ==> sum == 22140
invariant i == 42 ==> sum == 23821
invariant i == 43 ==> sum == 25585
invariant i == 44 ==> sum == 27434
invariant i == 45 ==> sum == 29370
invariant i == 46 ==> sum == 31395
invariant i == 47 ==> sum == 33511
invariant i == 48 ==> sum == 35720
invariant i == 49 ==> sum == 38024
invariant i == 50 ==> sum == 40425
invariant i == 51 ==> sum == 42925
invariant i == 52 ==> sum == 45526
invariant i == 53 ==> sum == 48230
invariant i == 54 ==> sum == 51039
invariant i == 55 ==> sum == 53955
invariant i == 56 ==> sum == 56980
invariant i == 57 ==> sum == 60116
invariant i == 58 ==> sum == 63365
invariant i == 59 ==> sum == 66729
invariant i == 60 ==> sum == 70210
invariant i == 61 ==> sum == 73810
invariant i == 62 ==> sum == 77531
invariant i == 63 ==> sum == 81375
invariant i == 64 ==> sum == 85344
invariant i == 65 ==> sum == 89440
invariant i == 66 ==> sum == 93665
```

```
    invariant i == 67 ==> sum == 98021
    invariant i == 68 ==> sum == 102510
    invariant i == 69 ==> sum == 107134
    invariant i == 70 ==> sum == 111895
    invariant i == 71 ==> sum == 116795
    invariant i == 72 ==> sum == 121836
    invariant i == 73 ==> sum == 127020
    invariant i == 74 ==> sum == 132349
    invariant i == 75 ==> sum == 137825
    invariant i == 76 ==> sum == 143450
    invariant i == 77 ==> sum == 149226
    invariant i == 78 ==> sum == 155155
    invariant i == 79 ==> sum == 161239
    invariant i == 80 ==> sum == 167480
    invariant i == 81 ==> sum == 173880
    invariant i == 82 ==> sum == 180441
    invariant i == 83 ==> sum == 187165
    invariant i == 84 ==> sum == 194054
    invariant i == 85 ==> sum == 201110
    invariant i == 86 ==> sum == 208335
    invariant i == 87 ==> sum == 215731
    invariant i == 88 ==> sum == 223300
    invariant i == 89 ==> sum == 231044
    invariant i == 90 ==> sum == 238965
    invariant i == 91 ==> sum == 247065
    invariant i == 92 ==> sum == 255346
    invariant i == 93 ==> sum == 263810
    invariant i == 94 ==> sum == 272459
    invariant i == 95 ==> sum == 281295
    invariant i == 96 ==> sum == 290320
    invariant i == 97 ==> sum == 299536
    invariant i == 98 ==> sum == 308945
    invariant i == 99 ==> sum == 318549
    invariant i == 100 ==> sum == 328350
    invariant i == 101 ==> sum == 338350
    decreases 101 - i
  {
    sum := sum + i * i;
    i := i + 1;
  }
}
```

Final Program

```
method p_4_1_b_sum_squares() returns (sum: int)
       ensures sum == 338350
{
  sum := 0;
  var i := 1;
  while i <= 100
```

```
invariant 1 <= i <= 101
invariant i == 1 ==> sum == 0
invariant i == 2 ==> sum == 1
invariant i == 3 ==> sum == 5
invariant i == 4 ==> sum == 14
invariant i == 5 ==> sum == 30
invariant i == 6 ==> sum == 55
invariant i == 7 ==> sum == 91
invariant i == 8 ==> sum == 140
invariant i == 9 ==> sum == 204
invariant i == 10 ==> sum == 285
invariant i == 11 ==> sum == 385
invariant i == 12 ==> sum == 506
invariant i == 13 ==> sum == 650
invariant i == 14 ==> sum == 819
invariant i == 15 ==> sum == 1015
invariant i == 16 ==> sum == 1240
invariant i == 17 ==> sum == 1496
invariant i == 18 ==> sum == 1785
invariant i == 19 ==> sum == 2109
invariant i == 20 ==> sum == 2470
invariant i == 21 ==> sum == 2870
invariant i == 22 ==> sum == 3311
invariant i == 23 ==> sum == 3795
invariant i == 24 ==> sum == 4324
invariant i == 25 ==> sum == 4900
invariant i == 26 ==> sum == 5525
invariant i == 27 ==> sum == 6201
invariant i == 28 ==> sum == 6930
invariant i == 29 ==> sum == 7714
invariant i == 30 ==> sum == 8555
invariant i == 31 ==> sum == 9455
invariant i == 32 ==> sum == 10416
invariant i == 33 ==> sum == 11440
invariant i == 34 ==> sum == 12529
invariant i == 35 ==> sum == 13685
invariant i == 36 ==> sum == 14910
invariant i == 37 ==> sum == 16206
invariant i == 38 ==> sum == 17575
invariant i == 39 ==> sum == 19019
invariant i == 40 ==> sum == 20540
invariant i == 41 ==> sum == 22140
invariant i == 42 ==> sum == 23821
invariant i == 43 ==> sum == 25585
invariant i == 44 ==> sum == 27434
invariant i == 45 ==> sum == 29370
invariant i == 46 ==> sum == 31395
invariant i == 47 ==> sum == 33511
invariant i == 48 ==> sum == 35720
invariant i == 49 ==> sum == 38024
invariant i == 50 ==> sum == 40425
```

```
invariant i == 51 ==> sum == 42925
invariant i == 52 ==> sum == 45526
invariant i == 53 ==> sum == 48230
invariant i == 54 ==> sum == 51039
invariant i == 55 ==> sum == 53955
invariant i == 56 ==> sum == 56980
invariant i == 57 ==> sum == 60116
invariant i == 58 ==> sum == 63365
invariant i == 59 ==> sum == 66729
invariant i == 60 ==> sum == 70210
invariant i == 61 ==> sum == 73810
invariant i == 62 ==> sum == 77531
invariant i == 63 ==> sum == 81375
invariant i == 64 ==> sum == 85344
invariant i == 65 ==> sum == 89440
invariant i == 66 ==> sum == 93665
invariant i == 67 ==> sum == 98021
invariant i == 68 ==> sum == 102510
invariant i == 69 ==> sum == 107134
invariant i == 70 ==> sum == 111895
invariant i == 71 ==> sum == 116795
invariant i == 72 ==> sum == 121836
invariant i == 73 ==> sum == 127020
invariant i == 74 ==> sum == 132349
invariant i == 75 ==> sum == 137825
invariant i == 76 ==> sum == 143450
invariant i == 77 ==> sum == 149226
invariant i == 78 ==> sum == 155155
invariant i == 79 ==> sum == 161239
invariant i == 80 ==> sum == 167480
invariant i == 81 ==> sum == 173880
invariant i == 82 ==> sum == 180441
invariant i == 83 ==> sum == 187165
invariant i == 84 ==> sum == 194054
invariant i == 85 ==> sum == 201110
invariant i == 86 ==> sum == 208335
invariant i == 87 ==> sum == 215731
invariant i == 88 ==> sum == 223300
invariant i == 89 ==> sum == 231044
invariant i == 90 ==> sum == 238965
invariant i == 91 ==> sum == 247065
invariant i == 92 ==> sum == 255346
invariant i == 93 ==> sum == 263810
invariant i == 94 ==> sum == 272459
invariant i == 95 ==> sum == 281295
invariant i == 96 ==> sum == 290320
invariant i == 97 ==> sum == 299536
invariant i == 98 ==> sum == 308945
invariant i == 99 ==> sum == 318549
invariant i == 100 ==> sum == 328350
invariant i == 101 ==> sum == 338350
```

```
    decreases 101 - i
  {
    sum := sum + i * i;
    i := i + 1;
  }
}
```

Total Token Usage

Input tokens: 266
Output tokens: 6552
Reasoning tokens: 5116
Sum of 'total tokens': 6818

Experiment Timings

Overall Experiment started at 1780485716674, ended at 1780485834685, lasting 118011ms (118.01 seconds)
Iteration #1 started at 1780485716675, ended at 1780485834684, lasting 118009ms (118.01 seconds)

Experiment 'p_4_1_c_powers_of_two_array' Results

June 3, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_4_1_c_powers_of_two_array
Natural language statement: Write a method with loops that computes all powers of 2 from 2^0 up to 2^20.
Method signature: p_4_1_c_powers_of_two_array() returns (powers: array<int>)

Ensures

- `powers.Length == 21`
- `forall i :: 0 <= i < powers.Length ==> powers[i] == pow(2, i)`

Functional Code Given

```
function pow(base: int, exp: int): int
  requires 0 <= exp
  decreases exp
{
  if exp == 0 then 1 else base * pow(base, exp - 1)
}
```

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method with loops that computes all powers of 2 from 2^0 up to 2^20.

The signature should be:

p_4_1_c_powers_of_two_array() returns (powers: array<int>)

The method should respect the following contract:

ensures powers.Length == 21, ensures forall i :: 0 <= i < powers.Length ==> powers[i] ==
    ↪ pow(2, i)

The contract uses the following dafny code:

function pow(base: int, exp: int): int
  requires 0 <= exp
  decreases exp
{
  if exp == 0 then 1 else base * pow(base, exp - 1)
}

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-Dme23bS6M65lsjFFxLdsduo5zV8eC
Model: gpt-5.5-2026-04-23
Created at: 1780485999
Finish reason: stop
Usage: 464 tokens in, and 3987 tokens out

```
{
  powers := new int[21];
  var i := 0;
  var p := 1;

  while i < 21
    invariant powers.Length == 21
    invariant 0 <= i <= 21
    invariant p == pow(2, i)
```

```
    invariant forall j :: 0 <= j < i ==> powers[j] == pow(2, j)
    decreases 21 - i
  {
    var oldI := i;
    var oldP := p;
    assert oldP == pow(2, oldI);

    powers[i] := p;
    assert powers[oldI] == pow(2, oldI);

    p := 2 * p;
    i := i + 1;

    assert i == oldI + 1;
    assert i > 0;
    assert i - 1 == oldI;
    assert pow(2, i) == 2 * pow(2, i - 1);
    assert p == 2 * oldP;
    assert p == pow(2, i);

    forall j | 0 <= j < i
      ensures powers[j] == pow(2, j)
    {
      if j == oldI {
        assert powers[j] == pow(2, j);
      } else {
        assert j < oldI + 1;
        assert j != oldI;
        assert j < oldI;
        assert powers[j] == pow(2, j);
      }
    }
  }
}
```

Final Program

```
function pow(base: int, exp: int): int
  requires 0 <= exp
  decreases exp
{
  if exp == 0 then 1 else base * pow(base, exp - 1)
}

method p_4_1_c_powers_of_two_array() returns (powers: array<int>)
       ensures powers.Length == 21
       ensures forall i :: 0 <= i < powers.Length ==> powers[i] == pow(2, i)
{
  powers := new int[21];
  var i := 0;
```

```
  var p := 1;

  while i < 21
    invariant powers.Length == 21
    invariant 0 <= i <= 21
    invariant p == pow(2, i)
    invariant forall j :: 0 <= j < i ==> powers[j] == pow(2, j)
    decreases 21 - i
  {
    var oldI := i;
    var oldP := p;
    assert oldP == pow(2, oldI);

    powers[i] := p;
    assert powers[oldI] == pow(2, oldI);

    p := 2 * p;
    i := i + 1;

    assert i == oldI + 1;
    assert i > 0;
    assert i - 1 == oldI;
    assert pow(2, i) == 2 * pow(2, i - 1);
    assert p == 2 * oldP;
    assert p == pow(2, i);

    forall j | 0 <= j < i
      ensures powers[j] == pow(2, j)
    {
      if j == oldI {
        assert powers[j] == pow(2, j);
      } else {
        assert j < oldI + 1;
        assert j != oldI;
        assert j < oldI;
        assert powers[j] == pow(2, j);
      }
    }
  }
}
```

Total Token Usage

Input tokens: 464
Output tokens: 3987
Reasoning tokens: 3582
Sum of 'total tokens': 4451

Experiment Timings

Overall Experiment started at 1780485999159, ended at 1780486077699, lasting 78540ms (78.54 seconds)
Iteration #1 started at 1780485999160, ended at 1780486077699, lasting 78539ms (78.54 seconds)

Experiment 'p_4_1_c_powers_of_two_seq' Results

June 3, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 2
Hard fails (resolution): 1
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_4_1_c_powers_of_two_seq
Natural language statement: Write a method with loops that computes all powers of 2 from 2^0 up to 2^20.
Method signature: p_4_1_c_powers_of_two_seq() returns (powers: seq<int>)

Ensures

- `|powers| == 21`
- `forall i :: 0 <= i < |powers| ==> powers[i] == pow(2, i)`

Functional Code Given

```
function pow(base: int, exp: int): int
  requires 0 <= exp
  decreases exp
{
  if exp == 0 then 1 else base * pow(base, exp - 1)
}
```

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method with loops that computes all powers of 2 from 2^0 up to 2^20.

The signature should be:

p_4_1_c_powers_of_two_seq() returns (powers: seq<int>)

The method should respect the following contract:

ensures |powers| == 21, ensures forall i :: 0 <= i < |powers| ==> powers[i] == pow(2, i)

The contract uses the following dafny code:

function pow(base: int, exp: int): int
  requires 0 <= exp
  decreases exp
{
  if exp == 0 then 1 else base * pow(base, exp - 1)
}

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DmdzQm4Gm323ZyAntEVP5DZFONwpe
Model: gpt-5.5-2026-04-23
Created at: 1780485836
Finish reason: stop
Usage: 468 tokens in, and 6615 tokens out

```
{
  powers := [];
  var i := 0;
  var current := 1;

  while i < 21
    invariant 0 <= i <= 21
    invariant |powers| == i
    invariant current == pow(2, i)
    invariant forall j :: 0 <= j < i ==> powers[j] == pow(2, j)
```

```
    decreases 21 - i
  {
    var nextI := i + 1;
    ghost var oldPowers := powers;

    powers := oldPowers + [current];
    assert |powers| == nextI;
    assert powers[i] == current;

    forall j | 0 <= j < nextI
      ensures powers[j] == pow(2, j)
    {
      if j < i {
        assert powers[j] == oldPowers[j];
      } else {
        assert j == i;
        assert powers[j] == current;
      }
    }

    current := current * 2;
    assert current == pow(2, i) * 2;
    assert current == 2 * pow(2, i);
    assert 0 < nextI;
    assert nextI - 1 == i;
    assert pow(2, nextI) == 2 * pow(2, nextI - 1);
    assert pow(2, nextI) == 2 * pow(2, i);
    assert current == pow(2, nextI);

    i := nextI;
  }

  assert i == 21;
  assert |powers| == 21;
  forall j | 0 <= j < |powers|
    ensures powers[j] == pow(2, j)
  {
    assert j < i;
  }
}
```

Program → GenAI

```
When using dafny resolve, the below error is emitted and resolve fails:

problem_data/p_4_1_c_powers_of_two_seq/program.dfy(26,14): Error: ghost variables such as
    ↪  oldPowers are allowed only in specification contexts. oldPowers was inferred to
    ↪ be ghost based on its declaration or initialization.
   |
26 | powers := oldPowers + [current];
   | ^^^^^^^^
```

```
1 resolution/type errors detected in program.dfy

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪  nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-Dme1TAGhRbg9B2VUwXpiiABStdXRX
Model: gpt-5.5-2026-04-23
Created at: 1780485963
Finish reason: stop
Usage: 980 tokens in, and 890 tokens out

```
{
  powers := [];
  var i := 0;
  var current := 1;

  while i < 21
    invariant 0 <= i <= 21
    invariant |powers| == i
    invariant current == pow(2, i)
    invariant forall j :: 0 <= j < i ==> powers[j] == pow(2, j)
    decreases 21 - i
  {
    var nextI := i + 1;
    var oldPowers := powers;

    powers := oldPowers + [current];
    assert |powers| == nextI;
    assert powers[i] == current;

    forall j | 0 <= j < nextI
      ensures powers[j] == pow(2, j)
    {
      if j < i {
        assert powers[j] == oldPowers[j];
      } else {
        assert j == i;
        assert powers[j] == current;
      }
    }

    current := current * 2;
    assert current == pow(2, i) * 2;
    assert current == 2 * pow(2, i);
    assert 0 < nextI;
    assert nextI - 1 == i;
```

```
    assert pow(2, nextI) == 2 * pow(2, nextI - 1);
    assert pow(2, nextI) == 2 * pow(2, i);
    assert current == pow(2, nextI);

    i := nextI;
  }

  assert i == 21;
  assert |powers| == 21;
  forall j | 0 <= j < |powers|
    ensures powers[j] == pow(2, j)
  {
    assert j < i;
  }
}
```

Final Program

```
function pow(base: int, exp: int): int
  requires 0 <= exp
  decreases exp
{
  if exp == 0 then 1 else base * pow(base, exp - 1)
}

method p_4_1_c_powers_of_two_seq() returns (powers: seq<int>)
       ensures |powers| == 21
       ensures forall i :: 0 <= i < |powers| ==> powers[i] == pow(2, i)
{
  powers := [];
  var i := 0;
  var current := 1;

  while i < 21
    invariant 0 <= i <= 21
    invariant |powers| == i
    invariant current == pow(2, i)
    invariant forall j :: 0 <= j < i ==> powers[j] == pow(2, j)
    decreases 21 - i
  {
    var nextI := i + 1;
    var oldPowers := powers;

    powers := oldPowers + [current];
    assert |powers| == nextI;
    assert powers[i] == current;

    forall j | 0 <= j < nextI
      ensures powers[j] == pow(2, j)
    {
```

```
    if j < i {
      assert powers[j] == oldPowers[j];
    } else {
      assert j == i;
      assert powers[j] == current;
    }
  }

  current := current * 2;
  assert current == pow(2, i) * 2;
  assert current == 2 * pow(2, i);
  assert 0 < nextI;
  assert nextI - 1 == i;
  assert pow(2, nextI) == 2 * pow(2, nextI - 1);
  assert pow(2, nextI) == 2 * pow(2, i);
  assert current == pow(2, nextI);

  i := nextI;
 }

 assert i == 21;
 assert |powers| == 21;
 forall j | 0 <= j < |powers|
  ensures powers[j] == pow(2, j)
 {
  assert j < i;
 }
}
```

Total Token Usage

Input tokens: 1448
Output tokens: 7505
Reasoning tokens: 6590
Sum of 'total tokens': 8953

Experiment Timings

Overall Experiment started at 1780485835810, ended at 1780485998124, lasting 162314ms (162.31 seconds)
Iteration #1 started at 1780485835811, ended at 1780485962951, lasting 127140ms (127.14 seconds)
Iteration #2 started at 1780485962956, ended at 1780485998123, lasting 35167ms (35.17 seconds)

Experiment 'p_4_1_d_sum_odd_numbers' Results

June 3, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_4_1_d_sum_odd_numbers
Natural language statement: Write a method with loops that computes the sum of all odd numbers between a and b (inclusive), where a and b are inputs.
Method signature: p_4_1_d_sum_odd_numbers(a: int, b: int) returns (sum: int)

Ensures

- `sum == sum_odd(a, b)`

Functional Code Given

```
function sum_odd(a: int, b: int): int
decreases b - a
{
  if a > b then 0
  else if a % 2 == 1 then a + sum_odd(a + 1, b)
  else sum_odd(a + 1, b)
}
```

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method with loops that computes the sum of all odd numbers between a and b (
    ↪ inclusive), where a and b are inputs.

The signature should be:

p_4_1_d_sum_odd_numbers(a: int, b: int) returns (sum: int)

The method should respect the following contract:

ensures sum == sum_odd(a, b)

The contract uses the following dafny code:

function sum_odd(a: int, b: int): int
decreases b - a
{
  if a > b then 0
  else if a % 2 == 1 then a + sum_odd(a + 1, b)
  else sum_odd(a + 1, b)
}

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-Dme3K67Y5MfVhc9nDeTk6WhDZoKVp
Model: gpt-5.5-2026-04-23
Created at: 1780486078
Finish reason: stop
Usage: 480 tokens in, and 1478 tokens out

```
{
  sum := 0;
  var i := a;

  while i <= b
    invariant sum + sum_odd(i, b) == sum_odd(a, b)
    decreases b - i + 1
  {
```

```
    if i % 2 == 1 {
      assert sum_odd(i, b) == i + sum_odd(i + 1, b);
      sum := sum + i;
    } else {
      assert sum_odd(i, b) == sum_odd(i + 1, b);
    }
    i := i + 1;
  }

  assert i > b;
  assert sum_odd(i, b) == 0;
}
```

Final Program

```
function sum_odd(a: int, b: int): int
decreases b - a
{
  if a > b then 0
  else if a % 2 == 1 then a + sum_odd(a + 1, b)
  else sum_odd(a + 1, b)
}

method p_4_1_d_sum_odd_numbers(a: int, b: int) returns (sum: int)
       ensures sum == sum_odd(a, b)
{
  sum := 0;
  var i := a;

  while i <= b
    invariant sum + sum_odd(i, b) == sum_odd(a, b)
    decreases b - i + 1
  {
    if i % 2 == 1 {
      assert sum_odd(i, b) == i + sum_odd(i + 1, b);
      sum := sum + i;
    } else {
      assert sum_odd(i, b) == sum_odd(i + 1, b);
    }
    i := i + 1;
  }

  assert i > b;
  assert sum_odd(i, b) == 0;
}
```

Total Token Usage

Input tokens: 480
Output tokens: 1478
Reasoning tokens: 1288
Sum of 'total tokens': 1958

Experiment Timings

Overall Experiment started at 1780486078518, ended at 1780486111241, lasting 32723ms (32.72 seconds)
Iteration #1 started at 1780486078518, ended at 1780486111241, lasting 32723ms (32.72 seconds)

Experiment 'p_4_1_e_sum_odd_digits' Results

June 3, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_4_1_e_sum_odd_digits
Natural language statement: Write a method with loops that computes the sum of all odd digits of an input. (For example, if the input is 32677, the sum would be 3 + 7 + 7 = 17.)
Method signature: p_4_1_e_sum_odd_digits(a: int) returns (sum: int)

Ensures

- `sum >= 0`
- `sum == s_o_d(a)`

Requires

- `a >= 0`

Functional Code Given

```
function s_o_d(a: int): int
requires a >= 0
{
  if a == 0 then 0
  else if a % 2 == 0 then s_o_d(a / 10)
  else a % 10 + s_o_d(a / 10)
}
```

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method with loops that computes the sum of all odd digits of an input. (For
    ↪ example, if the input is 32677, the sum would be 3 + 7 + 7 = 17.)

The signature should be:

p_4_1_e_sum_odd_digits(a: int) returns (sum: int)

The method should respect the following contract:

requires a >= 0, ensures sum >= 0, ensures sum == s_o_d(a)

The contract uses the following dafny code:

function s_o_d(a: int): int
requires a >= 0
{
  if a == 0 then 0
  else if a % 2 == 0 then s_o_d(a / 10)
  else a % 10 + s_o_d(a / 10)
}

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-Dme3swDYAesF9z9CEgFx10Irq5GkN
Model: gpt-5.5-2026-04-23
Created at: 1780486112
Finish reason: stop
Usage: 518 tokens in, and 2783 tokens out

```
{
  var x := a;
  sum := 0;
  while x > 0
    invariant x >= 0
    invariant sum >= 0
    invariant sum + s_o_d(x) == s_o_d(a)
    decreases x
```

```
  {
    if x % 2 == 0 {
      assert s_o_d(x) == s_o_d(x / 10);
      x := x / 10;
    } else {
      assert x % 10 >= 0;
      assert s_o_d(x) == x % 10 + s_o_d(x / 10);
      sum := sum + x % 10;
      x := x / 10;
    }
  }
  assert x == 0;
  assert s_o_d(x) == 0;
}
```

Final Program

```
function s_o_d(a: int): int
requires a >= 0
{
  if a == 0 then 0
  else if a % 2 == 0 then s_o_d(a / 10)
  else a % 10 + s_o_d(a / 10)
}

method p_4_1_e_sum_odd_digits(a: int) returns (sum: int)
       requires a >= 0
       ensures sum >= 0
       ensures sum == s_o_d(a)
{
  var x := a;
  sum := 0;
  while x > 0
    invariant x >= 0
    invariant sum >= 0
    invariant sum + s_o_d(x) == s_o_d(a)
    decreases x
  {
    if x % 2 == 0 {
      assert s_o_d(x) == s_o_d(x / 10);
      x := x / 10;
    } else {
      assert x % 10 >= 0;
      assert s_o_d(x) == x % 10 + s_o_d(x / 10);
      sum := sum + x % 10;
      x := x / 10;
    }
  }
  assert x == 0;
  assert s_o_d(x) == 0;
```

```
}
```

Total Token Usage

Input tokens: 518
Output tokens: 2783
Reasoning tokens: 2559
Sum of 'total tokens': 3301

Experiment Timings

Overall Experiment started at 1780486112028, ended at 1780486170026, lasting 57998ms (58.00 seconds)
Iteration #1 started at 1780486112034, ended at 1780486170026, lasting 57992ms (57.99 seconds)

Experiment 'p_4_2_a_min_max_array' Results

June 3, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_4_2_a_min_max_array
Natural language statement: Write a method that takes a sequence of integers and returns the smallest and largest of the inputs.
Method signature: p_4_2_a_min_max_array(inputs: array<int>) returns (min: int, max: int)

Ensures

- `forall i :: 0 <= i < inputs.Length ==> min <= inputs[i]`
- `forall i :: 0 <= i < inputs.Length ==> max >= inputs[i]`
- `exists i :: 0 <= i < inputs.Length && min == inputs[i]`
- `exists i :: 0 <= i < inputs.Length && max == inputs[i]`

Requires

- `0 < inputs.Length`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method that takes a sequence of integers and returns the smallest and largest of
    ↪ the inputs.

The signature should be:

p_4_2_a_min_max_array(inputs: array<int>) returns (min: int, max: int)

The method should respect the following contract:

requires 0 < inputs.Length, ensures forall i :: 0 <= i < inputs.Length ==> min <= inputs[
    ↪ i], ensures forall i :: 0 <= i < inputs.Length ==> max >= inputs[i], ensures
    ↪ exists i :: 0 <= i < inputs.Length && min == inputs[i], ensures exists i :: 0 <= i
    ↪  < inputs.Length && max == inputs[i]

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-Dme5ULJ9LZgJbBLvibBGpoXu7keZu
Model: gpt-5.5-2026-04-23
Created at: 1780486212
Finish reason: stop
Usage: 412 tokens in, and 588 tokens out

```
{
  min := inputs[0];
  max := inputs[0];

  var i := 1;
  while i < inputs.Length
    invariant 1 <= i <= inputs.Length
    invariant forall j :: 0 <= j < i ==> min <= inputs[j]
    invariant forall j :: 0 <= j < i ==> max >= inputs[j]
    invariant exists j :: 0 <= j < i && min == inputs[j]
    invariant exists j :: 0 <= j < i && max == inputs[j]
  {
    if inputs[i] < min {
      min := inputs[i];
    }
    if inputs[i] > max {
      max := inputs[i];
```

```
    }
    i := i + 1;
  }
}
```

Final Program

```
method p_4_2_a_min_max_array(inputs: array<int>) returns (min: int, max: int)
       requires 0 < inputs.Length
       ensures forall i :: 0 <= i < inputs.Length ==> min <= inputs[i]
       ensures forall i :: 0 <= i < inputs.Length ==> max >= inputs[i]
       ensures exists i :: 0 <= i < inputs.Length && min == inputs[i]
       ensures exists i :: 0 <= i < inputs.Length && max == inputs[i]
{
  min := inputs[0];
  max := inputs[0];

  var i := 1;
  while i < inputs.Length
    invariant 1 <= i <= inputs.Length
    invariant forall j :: 0 <= j < i ==> min <= inputs[j]
    invariant forall j :: 0 <= j < i ==> max >= inputs[j]
    invariant exists j :: 0 <= j < i && min == inputs[j]
    invariant exists j :: 0 <= j < i && max == inputs[j]
  {
    if inputs[i] < min {
      min := inputs[i];
    }
    if inputs[i] > max {
      max := inputs[i];
    }
    i := i + 1;
  }
}
```

Total Token Usage

Input tokens: 412
Output tokens: 588
Reasoning tokens: 397
Sum of 'total tokens': 1000

Experiment Timings

Overall Experiment started at 1780486211817, ended at 1780486227458, lasting 15641ms (15.64 seconds)
Iteration #1 started at 1780486211820, ended at 1780486227456, lasting 15636ms (15.64 seconds)

Experiment 'p_4_2_a_min_max_seq' Results

June 3, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_4_2_a_min_max_seq
Natural language statement: Write a method that takes a sequence of integers and returns the smallest and largest of the inputs.
Method signature: p_4_2_a_min_max_seq(inputs: seq<int>) returns (min: int, max: int)

Ensures

- `forall i :: 0 <= i < |inputs| ==> min <= inputs[i]`
- `forall i :: 0 <= i < |inputs| ==> max >= inputs[i]`
- `exists i :: 0 <= i < |inputs| && min == inputs[i]`
- `exists i :: 0 <= i < |inputs| && max == inputs[i]`

Requires

- `0 < |inputs|`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method that takes a sequence of integers and returns the smallest and largest of
    ↪ the inputs.

The signature should be:

p_4_2_a_min_max_seq(inputs: seq<int>) returns (min: int, max: int)

The method should respect the following contract:

requires 0 < |inputs|, ensures forall i :: 0 <= i < |inputs| ==> min <= inputs[i],
    ↪ ensures forall i :: 0 <= i < |inputs| ==> max >= inputs[i], ensures exists i :: 0
    ↪ <= i < |inputs| && min == inputs[i], ensures exists i :: 0 <= i < |inputs| && max
    ↪ == inputs[i]

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-Dme4pWxueOvzeC3iHYBG7QaT8afNw
Model: gpt-5.5-2026-04-23
Created at: 1780486171
Finish reason: stop
Usage: 422 tokens in, and 1613 tokens out

```
{
  min := inputs[0];
  max := inputs[0];

  ghost var minIndex := 0;
  ghost var maxIndex := 0;
  var i := 1;

  while i < |inputs|
    invariant 1 <= i <= |inputs|
    invariant 0 <= minIndex < i
    invariant 0 <= maxIndex < i
    invariant min == inputs[minIndex]
    invariant max == inputs[maxIndex]
    invariant forall j :: 0 <= j < i ==> min <= inputs[j]
    invariant forall j :: 0 <= j < i ==> max >= inputs[j]
    decreases |inputs| - i
```

```
  {
    if inputs[i] < min {
      min := inputs[i];
      minIndex := i;
    }

    if inputs[i] > max {
      max := inputs[i];
      maxIndex := i;
    }

    i := i + 1;
  }
}
```

Final Program

```
method p_4_2_a_min_max_seq(inputs: seq<int>) returns (min: int, max: int)
       requires 0 < |inputs|
       ensures forall i :: 0 <= i < |inputs| ==> min <= inputs[i]
       ensures forall i :: 0 <= i < |inputs| ==> max >= inputs[i]
       ensures exists i :: 0 <= i < |inputs| && min == inputs[i]
       ensures exists i :: 0 <= i < |inputs| && max == inputs[i]
{
  min := inputs[0];
  max := inputs[0];

  ghost var minIndex := 0;
  ghost var maxIndex := 0;
  var i := 1;

  while i < |inputs|
    invariant 1 <= i <= |inputs|
    invariant 0 <= minIndex < i
    invariant 0 <= maxIndex < i
    invariant min == inputs[minIndex]
    invariant max == inputs[maxIndex]
    invariant forall j :: 0 <= j < i ==> min <= inputs[j]
    invariant forall j :: 0 <= j < i ==> max >= inputs[j]
    decreases |inputs| - i
  {
    if inputs[i] < min {
      min := inputs[i];
      minIndex := i;
    }

    if inputs[i] > max {
      max := inputs[i];
      maxIndex := i;
    }
```

```
    i := i + 1;
  }
}
```

Total Token Usage

Input tokens: 422
Output tokens: 1613
Reasoning tokens: 1361
Sum of 'total tokens': 2035

Experiment Timings

Overall Experiment started at 1780486170881, ended at 1780486210996, lasting 40115ms (40.12 seconds)
Iteration #1 started at 1780486170888, ended at 1780486210996, lasting 40108ms (40.11 seconds)

Experiment 'p_4_2_b_count_even_odd_array' Results

June 3, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_4_2_b_count_even_odd_array
Natural language statement: Write a method that takes a sequence of integers and returns the number of even and odd inputs.
Method signature: p_4_2_b_count_even_odd_array(inputs: array<int>) returns (even_count: int, odd_count: int)

Ensures

- `even_count == number_even(inputs[..])`
- `odd_count == number_odd(inputs[..])`

Functional Code Given

```
function number_odd(s: seq<int>): int
{ if s == [] then 0
  else if s[0] % 2 == 1 then 1 + number_odd(s[1..])
  else number_odd(s[1..])
}

function number_even (s: seq<int>): int
{ if s == [] then 0
  else if s[0] % 2 == 0 then 1 + number_even(s[1..])
  else number_even(s[1..])
}
```

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method that takes a sequence of integers and returns the number of even and odd
    ↪ inputs.

The signature should be:

p_4_2_b_count_even_odd_array(inputs: array<int>) returns (even_count: int, odd_count: int
    ↪ )

The method should respect the following contract:

ensures even_count == number_even(inputs[..]), ensures odd_count == number_odd(inputs
    ↪ [..])

The contract uses the following dafny code:

function number_odd(s: seq<int>): int
{ if s == [] then 0
  else if s[0] % 2 == 1 then 1 + number_odd(s[1..])
  else number_odd(s[1..])
}

function number_even (s: seq<int>): int
{ if s == [] then 0
  else if s[0] % 2 == 0 then 1 + number_even(s[1..])
  else number_even(s[1..])
}

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-Dme62fGSpHMVJftUWY5AULOp7kNPX
Model: gpt-5.5-2026-04-23
Created at: 1780486246
Finish reason: stop
Usage: 578 tokens in, and 4495 tokens out

```
{
  even_count := 0;
```

```
  odd_count := 0;
  var i := inputs.Length;
  assert inputs[i..] == [];

  while i > 0
    invariant 0 <= i <= inputs.Length
    invariant even_count == number_even(inputs[i..])
    invariant odd_count == number_odd(inputs[i..])
    decreases i
  {
    ghost var tail := inputs[i..];
    i := i - 1;

    assert 0 <= i < inputs.Length;
    assert tail == inputs[i + 1..];
    assert inputs[i..i + 1] == [inputs[i]];
    assert inputs[i..] == inputs[i..i + 1] + inputs[i + 1..];
    assert inputs[i..] == [inputs[i]] + tail;
    assert inputs[i..] != [];
    assert (inputs[i..])[0] == inputs[i];
    assert (inputs[i..])[1..] == tail;

    if inputs[i] % 2 == 0 {
      assert inputs[i] % 2 != 1;
      even_count := even_count + 1;
      assert number_even(inputs[i..]) == 1 + number_even(tail);
      assert number_odd(inputs[i..]) == number_odd(tail);
    } else {
      assert 0 <= inputs[i] % 2 < 2;
      assert inputs[i] % 2 == 1;
      odd_count := odd_count + 1;
      assert number_even(inputs[i..]) == number_even(tail);
      assert number_odd(inputs[i..]) == 1 + number_odd(tail);
    }
  }

  assert i == 0;
  assert inputs[i..] == inputs[..];
}
```

Final Program

```
function number_odd(s: seq<int>): int
{ if s == [] then 0
  else if s[0] % 2 == 1 then 1 + number_odd(s[1..])
  else number_odd(s[1..])
}

function number_even (s: seq<int>): int
{ if s == [] then 0
```

```
  else if s[0] % 2 == 0 then 1 + number_even(s[1..])
  else number_even(s[1..])
}

method p_4_2_b_count_even_odd_array(inputs: array<int>) returns (even_count: int,
    ↪ odd_count: int)
       ensures even_count == number_even(inputs[..])
       ensures odd_count == number_odd(inputs[..])
{
  even_count := 0;
  odd_count := 0;
  var i := inputs.Length;
  assert inputs[i..] == [];

  while i > 0
    invariant 0 <= i <= inputs.Length
    invariant even_count == number_even(inputs[i..])
    invariant odd_count == number_odd(inputs[i..])
    decreases i
  {
    ghost var tail := inputs[i..];
    i := i - 1;

    assert 0 <= i < inputs.Length;
    assert tail == inputs[i + 1..];
    assert inputs[i..i + 1] == [inputs[i]];
    assert inputs[i..] == inputs[i..i + 1] + inputs[i + 1..];
    assert inputs[i..] == [inputs[i]] + tail;
    assert inputs[i..] != [];
    assert (inputs[i..])[0] == inputs[i];
    assert (inputs[i..])[1..] == tail;

    if inputs[i] % 2 == 0 {
      assert inputs[i] % 2 != 1;
      even_count := even_count + 1;
      assert number_even(inputs[i..]) == 1 + number_even(tail);
      assert number_odd(inputs[i..]) == number_odd(tail);
    } else {
      assert 0 <= inputs[i] % 2 < 2;
      assert inputs[i] % 2 == 1;
      odd_count := odd_count + 1;
      assert number_even(inputs[i..]) == number_even(tail);
      assert number_odd(inputs[i..]) == 1 + number_odd(tail);
    }
  }

  assert i == 0;
  assert inputs[i..] == inputs[..];
}
```

Total Token Usage

Input tokens: 578
Output tokens: 4495
Reasoning tokens: 4026
Sum of 'total tokens': 5073

Experiment Timings

Overall Experiment started at 1780486246470, ended at 1780486331221, lasting 84751ms (84.75 seconds)
Iteration #1 started at 1780486246471, ended at 1780486331221, lasting 84750ms (84.75 seconds)

Experiment 'p_4_2_b_count_even_odd_seq' Results

June 3, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_4_2_b_count_even_odd_seq
Natural language statement: Write a method that takes a sequence of integers and returns the number of even and odd inputs.
Method signature: p_4_2_b_count_even_odd_seq(inputs: seq<int>) returns (even_count: int, odd_count: int)

Ensures

- `even_count == number_even(inputs)`
- `odd_count == number_odd(inputs)`

Functional Code Given

```
function number_odd(s: seq<int>): int
{ if s == [] then 0
  else if s[0] % 2 == 1 then 1 + number_odd(s[1..])
  else number_odd(s[1..])
}

function number_even (s: seq<int>): int
{ if s == [] then 0
  else if s[0] % 2 == 0 then 1 + number_even(s[1..])
  else number_even(s[1..])
}
```

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method that takes a sequence of integers and returns the number of even and odd
    ↪ inputs.

The signature should be:

p_4_2_b_count_even_odd_seq(inputs: seq<int>) returns (even_count: int, odd_count: int)

The method should respect the following contract:

ensures even_count == number_even(inputs), ensures odd_count == number_odd(inputs)

The contract uses the following dafny code:

function number_odd(s: seq<int>): int
{ if s == [] then 0
  else if s[0] % 2 == 1 then 1 + number_odd(s[1..])
  else number_odd(s[1..])
}

function number_even (s: seq<int>): int
{ if s == [] then 0
  else if s[0] % 2 == 0 then 1 + number_even(s[1..])
  else number_even(s[1..])
}

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-Dme5kB14evLzNLjfgisogQkBJDFts
Model: gpt-5.5-2026-04-23
Created at: 1780486228
Finish reason: stop
Usage: 574 tokens in, and 643 tokens out

```
{
  if inputs == [] {
    even_count := 0;
    odd_count := 0;
```

```
  } else {
    var e, o := p_4_2_b_count_even_odd_seq(inputs[1..]);
    if inputs[0] % 2 == 0 {
      even_count := e + 1;
      odd_count := o;
    } else {
      even_count := e;
      odd_count := o + 1;
    }
  }
}
```

Final Program

```
function number_odd(s: seq<int>): int
{ if s == [] then 0
  else if s[0] % 2 == 1 then 1 + number_odd(s[1..])
  else number_odd(s[1..])
}

function number_even (s: seq<int>): int
{ if s == [] then 0
  else if s[0] % 2 == 0 then 1 + number_even(s[1..])
  else number_even(s[1..])
}

method p_4_2_b_count_even_odd_seq(inputs: seq<int>) returns (even_count: int, odd_count:
    ↪ int)
       ensures even_count == number_even(inputs)
       ensures odd_count == number_odd(inputs)
{
  if inputs == [] {
    even_count := 0;
    odd_count := 0;
  } else {
    var e, o := p_4_2_b_count_even_odd_seq(inputs[1..]);
    if inputs[0] % 2 == 0 {
      even_count := e + 1;
      odd_count := o;
    } else {
      even_count := e;
      odd_count := o + 1;
    }
  }
}
```

Total Token Usage

Input tokens: 574
Output tokens: 643
Reasoning tokens: 512
Sum of 'total tokens': 1217

Experiment Timings

Overall Experiment started at 1780486228144, ended at 1780486245777, lasting 17633ms (17.63 seconds)
Iteration #1 started at 1780486228145, ended at 1780486245777, lasting 17632ms (17.63 seconds)

Experiment 'p_4_2_c_cumulative_totals_array' Results

June 3, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_4_2_c_cumulative_totals_array
Natural language statement: Write a method that takes a sequence of integers and returns cumulative totals. For example, if the input is [1, 7, 2, 9], the method should return [1, 8, 10, 19].
Method signature: p_4_2_c_cumulative_totals_array(inputs: array<int>) returns (totals: array<int>)

Ensures

- `totals.Length == inputs.Length`
- `forall i :: 0 <= i < inputs.Length ==> totals[i] == (if i == 0 then inputs[0] else totals[i - 1] + inputs[i])`

Requires

- `0 < inputs.Length`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method that takes a sequence of integers and returns cumulative totals. For
    ↪ example, if the input is [1, 7, 2, 9], the method should return [1, 8, 10, 19].

The signature should be:

p_4_2_c_cumulative_totals_array(inputs: array<int>) returns (totals: array<int>)

The method should respect the following contract:

requires 0 < inputs.Length, ensures totals.Length == inputs.Length, ensures forall i :: 0
    ↪  <= i < inputs.Length ==> totals[i] == (if i == 0 then inputs[0] else totals[i -
    ↪ 1] + inputs[i])

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-Dme8v8f95SahOYIM4e8KSAmwXgf0Q
Model: gpt-5.5-2026-04-23
Created at: 1780486425
Finish reason: stop
Usage: 428 tokens in, and 660 tokens out

```
{
  totals := new int[inputs.Length];
  totals[0] := inputs[0];

  var i := 1;
  while i < inputs.Length
    invariant 1 <= i <= inputs.Length
    invariant totals.Length == inputs.Length
    invariant totals[0] == inputs[0]
    invariant forall j :: 1 <= j < i ==> totals[j] == totals[j - 1] + inputs[j]
  {
    totals[i] := totals[i - 1] + inputs[i];
    i := i + 1;
  }
}
```

Final Program

```
method p_4_2_c_cumulative_totals_array(inputs: array<int>) returns (totals: array<int>)
       requires 0 < inputs.Length
       ensures totals.Length == inputs.Length
       ensures forall i :: 0 <= i < inputs.Length ==> totals[i] == (if i == 0 then inputs
           ↪ [0] else totals[i - 1] + inputs[i])
{
  totals := new int[inputs.Length];
  totals[0] := inputs[0];

  var i := 1;
  while i < inputs.Length
    invariant 1 <= i <= inputs.Length
    invariant totals.Length == inputs.Length
    invariant totals[0] == inputs[0]
    invariant forall j :: 1 <= j < i ==> totals[j] == totals[j - 1] + inputs[j]
  {
    totals[i] := totals[i - 1] + inputs[i];
    i := i + 1;
  }
}
```

Total Token Usage

Input tokens: 428
Output tokens: 660
Reasoning tokens: 512
Sum of 'total tokens': 1088

Experiment Timings

Overall Experiment started at 1780486425389, ended at 1780486443180, lasting 17791ms (17.79 seconds)
Iteration #1 started at 1780486425390, ended at 1780486443180, lasting 17790ms (17.79 seconds)

Experiment 'p_4_2_c_cumulative_totals_seq' Results

June 3, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_4_2_c_cumulative_totals_seq
Natural language statement: Write a method that takes a sequence of integers and returns cumulative totals. For example, if the input is [1, 7, 2, 9], the method should return [1, 8, 10, 19].
Method signature: p_4_2_c_cumulative_totals_seq(inputs: seq<int>) returns (totals: seq<int>)

Ensures

- `|totals| == |inputs|`
- `forall i :: 0 <= i < |inputs| ==> totals[i] == (if i == 0 then inputs[0] else totals[i - 1] + inputs[i])`

Requires

- `0 < |inputs|`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method that takes a sequence of integers and returns cumulative totals. For
    ↪ example, if the input is [1, 7, 2, 9], the method should return [1, 8, 10, 19].

The signature should be:

p_4_2_c_cumulative_totals_seq(inputs: seq<int>) returns (totals: seq<int>)

The method should respect the following contract:

requires 0 < |inputs|, ensures |totals| == |inputs|, ensures forall i :: 0 <= i < |inputs
    ↪ | ==> totals[i] == (if i == 0 then inputs[0] else totals[i - 1] + inputs[i])

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-Dme7Q8IQJwe3DgJ2dE3oKxfczQWOm
Model: gpt-5.5-2026-04-23
Created at: 1780486332
Finish reason: stop
Usage: 438 tokens in, and 5178 tokens out

```
{
  totals := [inputs[0]];
  assert |totals| == 1;
  assert totals[0] == inputs[0];
  assert forall j :: 0 <= j < 1 ==> totals[j] == (if j == 0 then inputs[0] else totals[j
      ↪ - 1] + inputs[j]) by {
    forall j | 0 <= j < 1
      ensures totals[j] == (if j == 0 then inputs[0] else totals[j - 1] + inputs[j])
    {
      assert j == 0;
      assert totals[j] == inputs[0];
    }
  }

  var i := 1;
  while i < |inputs|
    invariant 1 <= i <= |inputs|
    invariant |totals| == i
```

```
  invariant forall j :: 0 <= j < i ==> totals[j] == (if j == 0 then inputs[0] else
      ↪ totals[j - 1] + inputs[j])
{
  var oldTotals := totals;
  assert |oldTotals| == i;

  totals := oldTotals + [oldTotals[i - 1] + inputs[i]];
  assert |totals| == i + 1;

  assert forall j :: 0 <= j < i + 1 ==> totals[j] == (if j == 0 then inputs[0] else
      ↪ totals[j - 1] + inputs[j]) by {
    forall j | 0 <= j < i + 1
      ensures totals[j] == (if j == 0 then inputs[0] else totals[j - 1] + inputs[j])
    {
      if j < i {
        assert 0 <= j < |oldTotals|;
        assert totals[j] == oldTotals[j];
        assert oldTotals[j] == (if j == 0 then inputs[0] else oldTotals[j - 1] + inputs[
            ↪ j]);
        if j == 0 {
          assert totals[j] == inputs[0];
        } else {
          assert 0 <= j - 1 < |oldTotals|;
          assert totals[j - 1] == oldTotals[j - 1];
          assert totals[j] == totals[j - 1] + inputs[j];
        }
      } else {
        assert j == i;
        assert j != 0;
        assert totals[j] == oldTotals[i - 1] + inputs[i];
        assert 0 <= i - 1 < |oldTotals|;
        assert totals[i - 1] == oldTotals[i - 1];
        assert j - 1 == i - 1;
        assert totals[j - 1] == oldTotals[i - 1];
        assert inputs[j] == inputs[i];
        assert totals[j] == totals[j - 1] + inputs[j];
      }
    }
  }

  i := i + 1;
}

assert i == |inputs|;
assert |totals| == |inputs|;
assert forall j :: 0 <= j < |inputs| ==> totals[j] == (if j == 0 then inputs[0] else
    ↪ totals[j - 1] + inputs[j]) by {
  forall j | 0 <= j < |inputs|
    ensures totals[j] == (if j == 0 then inputs[0] else totals[j - 1] + inputs[j])
  {
    assert j < i;
```

```
    }
  }
}
```

Final Program

```
method p_4_2_c_cumulative_totals_seq(inputs: seq<int>) returns (totals: seq<int>)
      requires 0 < |inputs|
      ensures |totals| == |inputs|
      ensures forall i :: 0 <= i < |inputs| ==> totals[i] == (if i == 0 then inputs[0]
          ↪ else totals[i - 1] + inputs[i])
{
  totals := [inputs[0]];
  assert |totals| == 1;
  assert totals[0] == inputs[0];
  assert forall j :: 0 <= j < 1 ==> totals[j] == (if j == 0 then inputs[0] else totals[j
      ↪ - 1] + inputs[j]) by {
    forall j | 0 <= j < 1
      ensures totals[j] == (if j == 0 then inputs[0] else totals[j - 1] + inputs[j])
    {
      assert j == 0;
      assert totals[j] == inputs[0];
    }
  }

  var i := 1;
  while i < |inputs|
    invariant 1 <= i <= |inputs|
    invariant |totals| == i
    invariant forall j :: 0 <= j < i ==> totals[j] == (if j == 0 then inputs[0] else
        ↪ totals[j - 1] + inputs[j])
  {
    var oldTotals := totals;
    assert |oldTotals| == i;

    totals := oldTotals + [oldTotals[i - 1] + inputs[i]];
    assert |totals| == i + 1;

    assert forall j :: 0 <= j < i + 1 ==> totals[j] == (if j == 0 then inputs[0] else
        ↪ totals[j - 1] + inputs[j]) by {
      forall j | 0 <= j < i + 1
        ensures totals[j] == (if j == 0 then inputs[0] else totals[j - 1] + inputs[j])
      {
        if j < i {
          assert 0 <= j < |oldTotals|;
          assert totals[j] == oldTotals[j];
          assert oldTotals[j] == (if j == 0 then inputs[0] else oldTotals[j - 1] + inputs[
              ↪ j]);
          if j == 0 {
            assert totals[j] == inputs[0];
```

```
        } else {
          assert 0 <= j - 1 < |oldTotals|;
          assert totals[j - 1] == oldTotals[j - 1];
          assert totals[j] == totals[j - 1] + inputs[j];
        }
      } else {
        assert j == i;
        assert j != 0;
        assert totals[j] == oldTotals[i - 1] + inputs[i];
        assert 0 <= i - 1 < |oldTotals|;
        assert totals[i - 1] == oldTotals[i - 1];
        assert j - 1 == i - 1;
        assert totals[j - 1] == oldTotals[i - 1];
        assert inputs[j] == inputs[i];
        assert totals[j] == totals[j - 1] + inputs[j];
      }
    }
  }

  i := i + 1;
 }

 assert i == |inputs|;
 assert |totals| == |inputs|;
 assert forall j :: 0 <= j < |inputs| ==> totals[j] == (if j == 0 then inputs[0] else
     ↪ totals[j - 1] + inputs[j]) by {
   forall j | 0 <= j < |inputs|
     ensures totals[j] == (if j == 0 then inputs[0] else totals[j - 1] + inputs[j])
   {
     assert j < i;
   }
 }
}
```

Total Token Usage

Input tokens: 438
Output tokens: 5178
Reasoning tokens: 4316
Sum of 'total tokens': 5616

Experiment Timings

Overall Experiment started at 1780486332187, ended at 1780486424364, lasting 92177ms (92.18 seconds)
Iteration #1 started at 1780486332188, ended at 1780486424364, lasting 92176ms (92.18 seconds)

Experiment 'p_4_2_d_adjacent_duplicates_array_without_4th_ensures' Results

June 4, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 2
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 2

Problem Specification

Problem name: p_4_2_d_adjacent_duplicates_array_without_4th_ensures
Natural language statement: Write a method that takes a sequence of integers and returns all adjacent duplicates. For example, if the input is [1, 3, 3, 4, 5, 5, 6, 6, 6, 2], the method should return [3, 5, 6].
Method signature: p_4_2_d_adjacent_duplicates_array_without_4th_ensures(inputs: array<int>) returns (duplicates: array<int>)

Ensures

- `forall d :: d in duplicates[..] ==> d in inputs[..]`
- `forall i :: 0 <= i < inputs.Length - 1 && inputs[i] == inputs[i + 1] ==> inputs[i] in duplicates[..]`
- `forall d :: d in duplicates[..] ==> exists i :: 0 <= i < inputs.Length - 1 && inputs[i] == inputs[i + 1] && inputs[i] == d`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method that takes a sequence of integers and returns all adjacent duplicates. For
    ↪  example, if the input is [1, 3, 3, 4, 5, 5, 6, 6, 6, 2], the method should return
    ↪  [3, 5, 6].

The signature should be:

p_4_2_d_adjacent_duplicates_array_without_4th_ensures(inputs: array<int>) returns (
    ↪ duplicates: array<int>)

The method should respect the following contract:

ensures forall d :: d in duplicates[..] ==> d in inputs[..], ensures forall i :: 0 <= i <
    ↪  inputs.Length - 1 && inputs[i] == inputs[i + 1] ==> inputs[i] in duplicates[..],
    ↪ ensures forall d :: d in duplicates[..] ==> exists i :: 0 <= i < inputs.Length - 1
    ↪  && inputs[i] == inputs[i + 1] && inputs[i] == d

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-Dn42CgTpAbtBpj6q5fKGSv2qktklu
Model: gpt-5.5-2026-04-23
Created at: 1780585952
Finish reason: stop
Usage: 546 tokens in, and 5654 tokens out

```
{
  var n := inputs.Length;
  var s: seq<int> := [];
  var i := 0;

  while i < n - 1
    invariant n == inputs.Length
    invariant 0 <= i <= n
    invariant forall d :: d in s ==> exists k :: 0 <= k < i && k < n - 1 && inputs[k] ==
        ↪ inputs[k + 1] && inputs[k] == d
    invariant forall k :: 0 <= k < i && k < n - 1 && inputs[k] == inputs[k + 1] ==> inputs
        ↪ [k] in s
  {
    var prev := s;
```

```
  if inputs[i] == inputs[i + 1] {
    s := s + [inputs[i]];
  }

  forall d | d in s
    ensures exists k :: 0 <= k < i + 1 && k < n - 1 && inputs[k] == inputs[k + 1] &&
        ↪ inputs[k] == d
  {
    if d in prev {
      var k :| 0 <= k < i && k < n - 1 && inputs[k] == inputs[k + 1] && inputs[k] == d;
      assert 0 <= k < i + 1;
    } else {
      if inputs[i] == inputs[i + 1] {
        assert s == prev + [inputs[i]];
        assert d == inputs[i];
        assert 0 <= i < i + 1;
        assert i < n - 1;
      } else {
        assert s == prev;
        assert d in prev;
      }
    }
  }

  forall k | 0 <= k < i + 1 && k < n - 1 && inputs[k] == inputs[k + 1]
    ensures inputs[k] in s
  {
    if k < i {
      assert inputs[k] in prev;
      if inputs[i] == inputs[i + 1] {
        assert s == prev + [inputs[i]];
        assert inputs[k] in s;
      } else {
        assert s == prev;
        assert inputs[k] in s;
      }
    } else {
      assert k == i;
      assert inputs[i] == inputs[i + 1];
      assert s == prev + [inputs[i]];
      assert inputs[k] == inputs[i];
      assert inputs[k] in s;
    }
  }

  i := i + 1;
}

duplicates := new int[|s|](j => s[j]);
assert duplicates.Length == |s|;
assert forall j :: 0 <= j < |s| ==> duplicates[j] == s[j];
```

```
  assert duplicates[..] == s;

  forall d | d in duplicates[..]
    ensures d in inputs[..]
  {
    assert d in s;
    var k :| 0 <= k < i && k < n - 1 && inputs[k] == inputs[k + 1] && inputs[k] == d;
    assert n == inputs.Length;
    assert 0 <= k < inputs.Length;
    assert inputs[..][k] == inputs[k];
    assert d in inputs[..];
  }

  forall k | 0 <= k < inputs.Length - 1 && inputs[k] == inputs[k + 1]
    ensures inputs[k] in duplicates[..]
  {
    assert n == inputs.Length;
    assert !(i < n - 1);
    assert i >= n - 1;
    assert k < n - 1;
    assert k < i;
    assert inputs[k] in s;
    assert inputs[k] in duplicates[..];
  }

  forall d | d in duplicates[..]
    ensures exists k :: 0 <= k < inputs.Length - 1 && inputs[k] == inputs[k + 1] && inputs
        ↪ [k] == d
  {
    assert d in s;
    var k :| 0 <= k < i && k < n - 1 && inputs[k] == inputs[k + 1] && inputs[k] == d;
    assert n == inputs.Length;
    assert 0 <= k < inputs.Length - 1 && inputs[k] == inputs[k + 1] && inputs[k] == d;
  }
}
```

Program → GenAI

```
When using dafny verify, the below error is emitted and verify fails:

problem_data/p_4_2_d_adjacent_duplicates_array_without_4th_ensures/program.dfy(65,35):
    ↪ Error: index out of range
   |
65 | duplicates := new int[|s|](j => s[j]);
   |                            ^


Dafny program verifier finished with 1 verified, 1 error

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
```

```
    ↪ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-Dn44GCIgnNDRA7XdQMqBZKIqGYy4U
Model: gpt-5.5-2026-04-23
Created at: 1780586080
Finish reason: stop
Usage: 1574 tokens in, and 4055 tokens out

```
{
  var n := inputs.Length;
  var s: seq<int> := [];
  var i := 0;

  while i < n - 1
    invariant n == inputs.Length
    invariant 0 <= i <= n
    invariant forall d :: d in s ==> exists k :: 0 <= k < i && k < n - 1 && inputs[k] ==
        ↪ inputs[k + 1] && inputs[k] == d
    invariant forall k :: 0 <= k < i && k < n - 1 && inputs[k] == inputs[k + 1] ==> inputs
        ↪ [k] in s
  {
    var prev := s;

    if inputs[i] == inputs[i + 1] {
      s := s + [inputs[i]];
    }

    forall d | d in s
      ensures exists k :: 0 <= k < i + 1 && k < n - 1 && inputs[k] == inputs[k + 1] &&
          ↪ inputs[k] == d
    {
      if d in prev {
        var k :| 0 <= k < i && k < n - 1 && inputs[k] == inputs[k + 1] && inputs[k] == d;
        assert 0 <= k < i + 1;
      } else {
        if inputs[i] == inputs[i + 1] {
          assert s == prev + [inputs[i]];
          assert d == inputs[i];
          assert 0 <= i < i + 1;
          assert i < n - 1;
        } else {
          assert s == prev;
          assert d in prev;
        }
      }
    }

    forall k | 0 <= k < i + 1 && k < n - 1 && inputs[k] == inputs[k + 1]
```

```
    ensures inputs[k] in s
  {
    if k < i {
      assert inputs[k] in prev;
      if inputs[i] == inputs[i + 1] {
        assert s == prev + [inputs[i]];
        assert inputs[k] in s;
      } else {
        assert s == prev;
        assert inputs[k] in s;
      }
    } else {
      assert k == i;
      assert inputs[i] == inputs[i + 1];
      assert s == prev + [inputs[i]];
      assert inputs[k] == inputs[i];
      assert inputs[k] in s;
    }
  }

  i := i + 1;
}

assert i >= n - 1;

duplicates := new int[|s|];
assert duplicates.Length == |s|;
assert duplicates != inputs;

var j := 0;
while j < |s|
  invariant n == inputs.Length
  invariant 0 <= i <= n
  invariant i >= n - 1
  invariant duplicates != inputs
  invariant duplicates.Length == |s|
  invariant 0 <= j <= |s|
  invariant duplicates[..j] == s[..j]
  invariant forall d :: d in s ==> exists k :: 0 <= k < i && k < n - 1 && inputs[k] ==
      ↪ inputs[k + 1] && inputs[k] == d
  invariant forall k :: 0 <= k < i && k < n - 1 && inputs[k] == inputs[k + 1] ==> inputs
      ↪ [k] in s
{
  duplicates[j] := s[j];
  assert duplicates[j] == s[j];
  assert duplicates[..j] == s[..j];
  assert duplicates[..j + 1] == s[..j + 1];
  j := j + 1;
}

assert j == |s|;
```

```
  assert duplicates[..] == duplicates[..j];
  assert s[..j] == s;
  assert duplicates[..] == s;

  forall d | d in duplicates[..]
    ensures d in inputs[..]
  {
    assert d in s;
    var k :| 0 <= k < i && k < n - 1 && inputs[k] == inputs[k + 1] && inputs[k] == d;
    assert n == inputs.Length;
    assert 0 <= k < inputs.Length;
    assert inputs[..][k] == inputs[k];
    assert d in inputs[..];
  }

  forall k | 0 <= k < inputs.Length - 1 && inputs[k] == inputs[k + 1]
    ensures inputs[k] in duplicates[..]
  {
    assert n == inputs.Length;
    assert !(i < n - 1);
    assert i >= n - 1;
    assert k < n - 1;
    assert k < i;
    assert inputs[k] in s;
    assert inputs[k] in duplicates[..];
  }

  forall d | d in duplicates[..]
    ensures exists k :: 0 <= k < inputs.Length - 1 && inputs[k] == inputs[k + 1] && inputs
        ↪ [k] == d
  {
    assert d in s;
    var k :| 0 <= k < i && k < n - 1 && inputs[k] == inputs[k + 1] && inputs[k] == d;
    assert n == inputs.Length;
    assert 0 <= k < inputs.Length - 1 && inputs[k] == inputs[k + 1] && inputs[k] == d;
  }
}
```

Final Program

```
method p_4_2_d_adjacent_duplicates_array_without_4th_ensures(inputs: array<int>) returns
    ↪ (duplicates: array<int>)
       ensures forall d :: d in duplicates[..] ==> d in inputs[..]
       ensures forall i :: 0 <= i < inputs.Length - 1 && inputs[i] == inputs[i + 1] ==>
           ↪ inputs[i] in duplicates[..]
       ensures forall d :: d in duplicates[..] ==> exists i :: 0 <= i < inputs.Length - 1
           ↪  && inputs[i] == inputs[i + 1] && inputs[i] == d
{
  var n := inputs.Length;
  var s: seq<int> := [];
```

```
var i := 0;

while i < n - 1
  invariant n == inputs.Length
  invariant 0 <= i <= n
  invariant forall d :: d in s ==> exists k :: 0 <= k < i && k < n - 1 && inputs[k] ==
      ↪ inputs[k + 1] && inputs[k] == d
  invariant forall k :: 0 <= k < i && k < n - 1 && inputs[k] == inputs[k + 1] ==> inputs
      ↪ [k] in s
{
  var prev := s;

  if inputs[i] == inputs[i + 1] {
    s := s + [inputs[i]];
  }

  forall d | d in s
    ensures exists k :: 0 <= k < i + 1 && k < n - 1 && inputs[k] == inputs[k + 1] &&
        ↪ inputs[k] == d
  {
    if d in prev {
      var k :| 0 <= k < i && k < n - 1 && inputs[k] == inputs[k + 1] && inputs[k] == d;
      assert 0 <= k < i + 1;
    } else {
      if inputs[i] == inputs[i + 1] {
        assert s == prev + [inputs[i]];
        assert d == inputs[i];
        assert 0 <= i < i + 1;
        assert i < n - 1;
      } else {
        assert s == prev;
        assert d in prev;
      }
    }
  }

  forall k | 0 <= k < i + 1 && k < n - 1 && inputs[k] == inputs[k + 1]
    ensures inputs[k] in s
  {
    if k < i {
      assert inputs[k] in prev;
      if inputs[i] == inputs[i + 1] {
        assert s == prev + [inputs[i]];
        assert inputs[k] in s;
      } else {
        assert s == prev;
        assert inputs[k] in s;
      }
    } else {
      assert k == i;
      assert inputs[i] == inputs[i + 1];
```

```
      assert s == prev + [inputs[i]];
      assert inputs[k] == inputs[i];
      assert inputs[k] in s;
    }
  }

  i := i + 1;
}

assert i >= n - 1;

duplicates := new int[|s|];
assert duplicates.Length == |s|;
assert duplicates != inputs;

var j := 0;
while j < |s|
  invariant n == inputs.Length
  invariant 0 <= i <= n
  invariant i >= n - 1
  invariant duplicates != inputs
  invariant duplicates.Length == |s|
  invariant 0 <= j <= |s|
  invariant duplicates[..j] == s[..j]
  invariant forall d :: d in s ==> exists k :: 0 <= k < i && k < n - 1 && inputs[k] ==
      ↪ inputs[k + 1] && inputs[k] == d
  invariant forall k :: 0 <= k < i && k < n - 1 && inputs[k] == inputs[k + 1] ==> inputs
      ↪ [k] in s
{
  duplicates[j] := s[j];
  assert duplicates[j] == s[j];
  assert duplicates[..j] == s[..j];
  assert duplicates[..j + 1] == s[..j + 1];
  j := j + 1;
}

assert j == |s|;
assert duplicates[..] == duplicates[..j];
assert s[..j] == s;
assert duplicates[..] == s;

forall d | d in duplicates[..]
  ensures d in inputs[..]
{
  assert d in s;
  var k :| 0 <= k < i && k < n - 1 && inputs[k] == inputs[k + 1] && inputs[k] == d;
  assert n == inputs.Length;
  assert 0 <= k < inputs.Length;
  assert inputs[..][k] == inputs[k];
  assert d in inputs[..];
}
```

```
  forall k | 0 <= k < inputs.Length - 1 && inputs[k] == inputs[k + 1]
    ensures inputs[k] in duplicates[..]
  {
    assert n == inputs.Length;
    assert !(i < n - 1);
    assert i >= n - 1;
    assert k < n - 1;
    assert k < i;
    assert inputs[k] in s;
    assert inputs[k] in duplicates[..];
  }

  forall d | d in duplicates[..]
    ensures exists k :: 0 <= k < inputs.Length - 1 && inputs[k] == inputs[k + 1] && inputs
        ↪ [k] == d
  {
    assert d in s;
    var k :| 0 <= k < i && k < n - 1 && inputs[k] == inputs[k + 1] && inputs[k] == d;
    assert n == inputs.Length;
    assert 0 <= k < inputs.Length - 1 && inputs[k] == inputs[k + 1] && inputs[k] == d;
  }
}
```

Total Token Usage

Input tokens: 2120
Output tokens: 9709
Reasoning tokens: 7386
Sum of 'total tokens': 11829

Experiment Timings

Overall Experiment started at 1780585951459, ended at 1780586194095, lasting 242636ms (242.64 seconds)
Iteration #1 started at 1780585951475, ended at 1780586079268, lasting 127793ms (127.79 seconds)
Iteration #2 started at 1780586079340, ended at 1780586194094, lasting 114754ms (114.75 seconds)

Experiment 'p_4_2_d_adjacent_duplicates_seq_without_4th_ensures' Results

June 4, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 2
Hard fails (resolution): 1
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_4_2_d_adjacent_duplicates_seq_without_4th_ensures
Natural language statement: Write a method that takes a sequence of integers and returns all adjacent duplicates. For example, if the input is [1, 3, 3, 4, 5, 5, 6, 6, 6, 2], the method should return [3, 5, 6].
Method signature: p_4_2_d_adjacent_duplicates_seq_without_4th_ensures(inputs: seq<int>) returns (duplicates: seq<int>)

Ensures

- `forall d :: d in duplicates ==> d in inputs`
- `forall i :: 0 <= i < |inputs| - 1 && inputs[i] == inputs[i + 1] ==> inputs[i] in duplicates`
- `forall d :: d in duplicates ==> exists i :: 0 <= i < |inputs| - 1 && inputs[i] == inputs[i + 1] && inputs[i] == d`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method that takes a sequence of integers and returns all adjacent duplicates. For
    ↪  example, if the input is [1, 3, 3, 4, 5, 5, 6, 6, 6, 2], the method should return
    ↪  [3, 5, 6].

The signature should be:

p_4_2_d_adjacent_duplicates_seq_without_4th_ensures(inputs: seq<int>) returns (duplicates
    ↪ : seq<int>)

The method should respect the following contract:

ensures forall d :: d in duplicates ==> d in inputs, ensures forall i :: 0 <= i < |inputs
    ↪ | - 1 && inputs[i] == inputs[i + 1] ==> inputs[i] in duplicates, ensures forall d
    ↪ :: d in duplicates ==> exists i :: 0 <= i < |inputs| - 1 && inputs[i] == inputs[i
    ↪ + 1] && inputs[i] == d

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-Dn3woUWxjHvHs7nwMKF7aWTkt6y3E
Model: gpt-5.5-2026-04-23
Created at: 1780585618
Finish reason: stop
Usage: 538 tokens in, and 8785 tokens out

```
{
  duplicates := [];
  var i := 0;
  while i < |inputs| - 1
    invariant 0 <= i <= |inputs|
    invariant forall d :: d in duplicates ==> d in inputs
    invariant forall j :: 0 <= j < i && j < |inputs| - 1 && inputs[j] == inputs[j + 1] ==>
        ↪  inputs[j] in duplicates
    invariant forall d :: d in duplicates ==> exists j :: 0 <= j < i && j < |inputs| - 1
        ↪ && inputs[j] == inputs[j + 1] && inputs[j] == d
    decreases |inputs| - i
  {
    var oldI := i;
    ghost var oldDuplicates := duplicates;
```

```
assert forall d :: d in oldDuplicates ==> d in inputs;
assert forall j :: 0 <= j < oldI && j < |inputs| - 1 && inputs[j] == inputs[j + 1] ==>
    ↪  inputs[j] in oldDuplicates;
assert forall d :: d in oldDuplicates ==> exists j :: 0 <= j < oldI && j < |inputs| -
    ↪ 1 && inputs[j] == inputs[j + 1] && inputs[j] == d;

if inputs[i] == inputs[i + 1] {
  if !(inputs[i] in duplicates) {
    assert 0 <= i < |inputs|;
    assert inputs[i] in inputs;
    duplicates := duplicates + [inputs[i]];
    assert duplicates == oldDuplicates + [inputs[oldI]];
    assert inputs[oldI] in duplicates;
  } else {
    assert inputs[oldI] in duplicates;
  }
}

assert duplicates == oldDuplicates || duplicates == oldDuplicates + [inputs[oldI]];
assert duplicates != oldDuplicates ==> duplicates == oldDuplicates + [inputs[oldI]];
assert duplicates != oldDuplicates ==> inputs[oldI] == inputs[oldI + 1];
assert inputs[oldI] == inputs[oldI + 1] ==> inputs[oldI] in duplicates;

assert forall d | d in oldDuplicates
  ensures d in duplicates
{
  if duplicates != oldDuplicates {
    assert duplicates == oldDuplicates + [inputs[oldI]];
    assert d in oldDuplicates + [inputs[oldI]];
  }
}

assert forall d | d in duplicates
  ensures d in inputs
{
  if d in oldDuplicates {
    assert d in inputs;
  } else {
    assert duplicates != oldDuplicates;
    assert duplicates == oldDuplicates + [inputs[oldI]];
    assert d in oldDuplicates + [inputs[oldI]];
    assert d in [inputs[oldI]];
    assert d == inputs[oldI];
    assert 0 <= oldI < |inputs|;
    assert inputs[oldI] in inputs;
  }
}

i := i + 1;
assert i == oldI + 1;
```

```
    assert forall j | 0 <= j < i && j < |inputs| - 1 && inputs[j] == inputs[j + 1]
      ensures inputs[j] in duplicates
    {
      if j < oldI {
        assert inputs[j] in oldDuplicates;
        assert inputs[j] in duplicates;
      } else {
        assert oldI <= j;
        assert j < oldI + 1;
        assert j == oldI;
        assert inputs[oldI] == inputs[oldI + 1];
        assert inputs[oldI] in duplicates;
      }
    }

    assert forall d | d in duplicates
      ensures exists j :: 0 <= j < i && j < |inputs| - 1 && inputs[j] == inputs[j + 1] &&
          ↪ inputs[j] == d
    {
      if d in oldDuplicates {
        assert exists j :: 0 <= j < oldI && j < |inputs| - 1 && inputs[j] == inputs[j + 1]
            ↪  && inputs[j] == d;
        ghost var k :| 0 <= k < oldI && k < |inputs| - 1 && inputs[k] == inputs[k + 1] &&
            ↪ inputs[k] == d;
        assert 0 <= k < i;
        assert k < |inputs| - 1;
        assert inputs[k] == inputs[k + 1];
        assert inputs[k] == d;
      } else {
        assert duplicates != oldDuplicates;
        assert duplicates == oldDuplicates + [inputs[oldI]];
        assert d in oldDuplicates + [inputs[oldI]];
        assert d in [inputs[oldI]];
        assert d == inputs[oldI];
        assert inputs[oldI] == inputs[oldI + 1];
        assert 0 <= oldI < i;
        assert oldI < |inputs| - 1;
      }
    }
  }

  assert forall j | 0 <= j < |inputs| - 1 && inputs[j] == inputs[j + 1]
    ensures inputs[j] in duplicates
  {
    assert !(i < |inputs| - 1);
    assert j < i;
  }
}
```

Program → GenAI

```
When using dafny resolve, the below error is emitted and resolve fails:

problem_data/p_4_2_d_adjacent_duplicates_seq_without_4th_ensures/program.dfy(64,6): Error
    ↪ : invalid AssertStmt
   |
64 | i := i + 1;
   | ^

1 parse errors detected in program.dfy

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪  nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-Dn3zf24y94DLGZrW5s8Yc56hZAJYs
Model: gpt-5.5-2026-04-23
Created at: 1780585795
Finish reason: stop
Usage: 1743 tokens in, and 5850 tokens out

```
{
  duplicates := [];
  var i := 0;
  while i < |inputs| - 1
    invariant 0 <= i <= |inputs|
    invariant forall d :: d in duplicates ==> d in inputs
    invariant forall j :: 0 <= j < i && j < |inputs| - 1 && inputs[j] == inputs[j + 1] ==>
        ↪  inputs[j] in duplicates
    invariant forall d :: d in duplicates ==> exists j :: 0 <= j < i && j < |inputs| - 1
        ↪ && inputs[j] == inputs[j + 1] && inputs[j] == d
    decreases |inputs| - i
  {
    var oldI := i;
    ghost var oldDuplicates := duplicates;

    assert forall d :: d in oldDuplicates ==> d in inputs;
    assert forall j :: 0 <= j < oldI && j < |inputs| - 1 && inputs[j] == inputs[j + 1] ==>
        ↪  inputs[j] in oldDuplicates;
    assert forall d :: d in oldDuplicates ==> exists j :: 0 <= j < oldI && j < |inputs| -
        ↪ 1 && inputs[j] == inputs[j + 1] && inputs[j] == d;

    if inputs[i] == inputs[i + 1] {
      if !(inputs[i] in duplicates) {
        assert 0 <= i < |inputs|;
        assert inputs[i] in inputs;
        duplicates := duplicates + [inputs[i]];
        assert duplicates == oldDuplicates + [inputs[oldI]];
        assert inputs[oldI] in duplicates;
```

```
  } else {
    assert inputs[oldI] in duplicates;
  }
}

assert duplicates == oldDuplicates || duplicates == oldDuplicates + [inputs[oldI]];
assert duplicates != oldDuplicates ==> duplicates == oldDuplicates + [inputs[oldI]];
assert duplicates != oldDuplicates ==> inputs[oldI] == inputs[oldI + 1];
assert inputs[oldI] == inputs[oldI + 1] ==> inputs[oldI] in duplicates;

forall d | d in oldDuplicates
  ensures d in duplicates
{
  if duplicates != oldDuplicates {
    assert duplicates == oldDuplicates + [inputs[oldI]];
    assert d in oldDuplicates + [inputs[oldI]];
  }
}

forall d | d in duplicates
  ensures d in inputs
{
  if d in oldDuplicates {
    assert d in inputs;
  } else {
    assert duplicates != oldDuplicates;
    assert duplicates == oldDuplicates + [inputs[oldI]];
    assert d in oldDuplicates + [inputs[oldI]];
    assert d in [inputs[oldI]];
    assert d == inputs[oldI];
    assert 0 <= oldI < |inputs|;
    assert inputs[oldI] in inputs;
  }
}

i := i + 1;
assert i == oldI + 1;

forall j | 0 <= j < i && j < |inputs| - 1 && inputs[j] == inputs[j + 1]
  ensures inputs[j] in duplicates
{
  if j < oldI {
    assert inputs[j] in oldDuplicates;
    assert inputs[j] in duplicates;
  } else {
    assert oldI <= j;
    assert j < oldI + 1;
    assert j == oldI;
    assert inputs[oldI] == inputs[oldI + 1];
    assert inputs[oldI] in duplicates;
  }
```

```
    }

    forall d | d in duplicates
      ensures exists j :: 0 <= j < i && j < |inputs| - 1 && inputs[j] == inputs[j + 1] &&
          ↪ inputs[j] == d
    {
      if d in oldDuplicates {
        assert exists j :: 0 <= j < oldI && j < |inputs| - 1 && inputs[j] == inputs[j + 1]
            ↪  && inputs[j] == d;
        ghost var k :| 0 <= k < oldI && k < |inputs| - 1 && inputs[k] == inputs[k + 1] &&
            ↪ inputs[k] == d;
        assert 0 <= k < i;
        assert k < |inputs| - 1;
        assert inputs[k] == inputs[k + 1];
        assert inputs[k] == d;
      } else {
        assert duplicates != oldDuplicates;
        assert duplicates == oldDuplicates + [inputs[oldI]];
        assert d in oldDuplicates + [inputs[oldI]];
        assert d in [inputs[oldI]];
        assert d == inputs[oldI];
        assert inputs[oldI] == inputs[oldI + 1];
        assert 0 <= oldI < i;
        assert oldI < |inputs| - 1;
      }
    }
  }

  forall j | 0 <= j < |inputs| - 1 && inputs[j] == inputs[j + 1]
    ensures inputs[j] in duplicates
  {
    assert !(i < |inputs| - 1);
    assert j < i;
  }
}
```

Final Program

```
method p_4_2_d_adjacent_duplicates_seq_without_4th_ensures(inputs: seq<int>) returns (
    ↪ duplicates: seq<int>)
      ensures forall d :: d in duplicates ==> d in inputs
      ensures forall i :: 0 <= i < |inputs| - 1 && inputs[i] == inputs[i + 1] ==> inputs
          ↪ [i] in duplicates
      ensures forall d :: d in duplicates ==> exists i :: 0 <= i < |inputs| - 1 &&
          ↪ inputs[i] == inputs[i + 1] && inputs[i] == d
{
  duplicates := [];
  var i := 0;
  while i < |inputs| - 1
    invariant 0 <= i <= |inputs|
```

```
  invariant forall d :: d in duplicates ==> d in inputs
  invariant forall j :: 0 <= j < i && j < |inputs| - 1 && inputs[j] == inputs[j + 1] ==>
      ↪  inputs[j] in duplicates
  invariant forall d :: d in duplicates ==> exists j :: 0 <= j < i && j < |inputs| - 1
      ↪ && inputs[j] == inputs[j + 1] && inputs[j] == d
  decreases |inputs| - i
{
  var oldI := i;
  ghost var oldDuplicates := duplicates;

  assert forall d :: d in oldDuplicates ==> d in inputs;
  assert forall j :: 0 <= j < oldI && j < |inputs| - 1 && inputs[j] == inputs[j + 1] ==>
      ↪  inputs[j] in oldDuplicates;
  assert forall d :: d in oldDuplicates ==> exists j :: 0 <= j < oldI && j < |inputs| -
      ↪ 1 && inputs[j] == inputs[j + 1] && inputs[j] == d;

  if inputs[i] == inputs[i + 1] {
    if !(inputs[i] in duplicates) {
      assert 0 <= i < |inputs|;
      assert inputs[i] in inputs;
      duplicates := duplicates + [inputs[i]];
      assert duplicates == oldDuplicates + [inputs[oldI]];
      assert inputs[oldI] in duplicates;
    } else {
      assert inputs[oldI] in duplicates;
    }
  }

  assert duplicates == oldDuplicates || duplicates == oldDuplicates + [inputs[oldI]];
  assert duplicates != oldDuplicates ==> duplicates == oldDuplicates + [inputs[oldI]];
  assert duplicates != oldDuplicates ==> inputs[oldI] == inputs[oldI + 1];
  assert inputs[oldI] == inputs[oldI + 1] ==> inputs[oldI] in duplicates;

  forall d | d in oldDuplicates
    ensures d in duplicates
  {
    if duplicates != oldDuplicates {
      assert duplicates == oldDuplicates + [inputs[oldI]];
      assert d in oldDuplicates + [inputs[oldI]];
    }
  }

  forall d | d in duplicates
    ensures d in inputs
  {
    if d in oldDuplicates {
      assert d in inputs;
    } else {
      assert duplicates != oldDuplicates;
      assert duplicates == oldDuplicates + [inputs[oldI]];
      assert d in oldDuplicates + [inputs[oldI]];
```

```
      assert d in [inputs[oldI]];
      assert d == inputs[oldI];
      assert 0 <= oldI < |inputs|;
      assert inputs[oldI] in inputs;
    }
  }

  i := i + 1;
  assert i == oldI + 1;

  forall j | 0 <= j < i && j < |inputs| - 1 && inputs[j] == inputs[j + 1]
    ensures inputs[j] in duplicates
  {
    if j < oldI {
      assert inputs[j] in oldDuplicates;
      assert inputs[j] in duplicates;
    } else {
      assert oldI <= j;
      assert j < oldI + 1;
      assert j == oldI;
      assert inputs[oldI] == inputs[oldI + 1];
      assert inputs[oldI] in duplicates;
    }
  }

  forall d | d in duplicates
    ensures exists j :: 0 <= j < i && j < |inputs| - 1 && inputs[j] == inputs[j + 1] &&
        ↪ inputs[j] == d
  {
    if d in oldDuplicates {
      assert exists j :: 0 <= j < oldI && j < |inputs| - 1 && inputs[j] == inputs[j + 1]
          ↪  && inputs[j] == d;
      ghost var k :| 0 <= k < oldI && k < |inputs| - 1 && inputs[k] == inputs[k + 1] &&
          ↪ inputs[k] == d;
      assert 0 <= k < i;
      assert k < |inputs| - 1;
      assert inputs[k] == inputs[k + 1];
      assert inputs[k] == d;
    } else {
      assert duplicates != oldDuplicates;
      assert duplicates == oldDuplicates + [inputs[oldI]];
      assert d in oldDuplicates + [inputs[oldI]];
      assert d in [inputs[oldI]];
      assert d == inputs[oldI];
      assert inputs[oldI] == inputs[oldI + 1];
      assert 0 <= oldI < i;
      assert oldI < |inputs| - 1;
    }
  }
}
```

```
  forall j | 0 <= j < |inputs| - 1 && inputs[j] == inputs[j + 1]
    ensures inputs[j] in duplicates
  {
    assert !(i < |inputs| - 1);
    assert j < i;
  }
}
```

Total Token Usage

Input tokens: 2281
Output tokens: 14635
Reasoning tokens: 12108
Sum of 'total tokens': 16916

Experiment Timings

Overall Experiment started at 1780585616264, ended at 1780585943367, lasting 327103ms (327.10 seconds)
Iteration #1 started at 1780585616271, ended at 1780585794648, lasting 178377ms (178.38 seconds)
Iteration #2 started at 1780585794655, ended at 1780585943367, lasting 148712ms (148.71 seconds)

Experiment 'p_4_3_a_extract_uppercase' Results

June 3, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 2
Hard fails (resolution): 1
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_4_3_a_extract_uppercase
Natural language statement: Write a method that takes a string and returns only the uppercase letters in the string.
Method signature: p_4_3_a_extract_uppercase(s: string) returns (uppercase: string)

Ensures

- `forall c :: c in uppercase ==> 'A' <= c <= 'Z'`
- `forall c :: c in s && 'A' <= c <= 'Z' ==> c in uppercase`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method that takes a string and returns only the uppercase letters in the string.

The signature should be:

p_4_3_a_extract_uppercase(s: string) returns (uppercase: string)

The method should respect the following contract:

ensures forall c :: c in uppercase ==> 'A' <= c <= 'Z', ensures forall c :: c in s && 'A'
    ↪  <= c <= 'Z' ==> c in uppercase

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-Dmei8sxeg7JxpAt1sT10fcFLSnwCm
Model: gpt-5.5-2026-04-23
Created at: 1780488608
Finish reason: stop
Usage: 328 tokens in, and 5510 tokens out

```
{
  uppercase := "";
  var i := 0;
  while i < |s|
    invariant 0 <= i <= |s|
    invariant forall c :: c in uppercase ==> 'A' <= c <= 'Z'
    invariant forall c :: c in s[..i] && 'A' <= c <= 'Z' ==> c in uppercase
    decreases |s| - i
  {
    var ch := s[i];
    assert |s[..i+1]| == i + 1;

    if 'A' <= ch <= 'Z' {
      var oldUppercase := uppercase;
      assert forall c :: c in oldUppercase ==> 'A' <= c <= 'Z';
      assert forall c :: c in s[..i] && 'A' <= c <= 'Z' ==> c in oldUppercase;

      uppercase := oldUppercase + [ch];
      assert |uppercase| == |oldUppercase| + 1;
```

```
    assert forall c | c in uppercase ensures 'A' <= c <= 'Z' {
      var j :| 0 <= j < |uppercase| && uppercase[j] == c;
      if j < |oldUppercase| {
        assert uppercase[j] == oldUppercase[j];
        assert oldUppercase[j] == c;
        assert c in oldUppercase;
      } else {
        assert j == |oldUppercase|;
        assert uppercase[j] == ch;
        assert c == ch;
      }
    }

    assert forall c | c in s[..i+1] && 'A' <= c <= 'Z' ensures c in uppercase {
      var j :| 0 <= j < |s[..i+1]| && (s[..i+1])[j] == c;
      if j < i {
        assert (s[..i+1])[j] == s[j];
        assert (s[..i])[j] == s[j];
        assert (s[..i])[j] == c;
        assert c in s[..i];
        assert c in oldUppercase;
        var k :| 0 <= k < |oldUppercase| && oldUppercase[k] == c;
        assert uppercase[k] == c;
        assert c in uppercase;
      } else {
        assert j == i;
        assert (s[..i+1])[j] == s[i];
        assert c == ch;
        assert uppercase[|oldUppercase|] == c;
        assert c in uppercase;
      }
    }
  } else {
    assert !('A' <= ch <= 'Z');
    assert forall c | c in s[..i+1] && 'A' <= c <= 'Z' ensures c in uppercase {
      var j :| 0 <= j < |s[..i+1]| && (s[..i+1])[j] == c;
      if j < i {
        assert (s[..i+1])[j] == s[j];
        assert (s[..i])[j] == s[j];
        assert (s[..i])[j] == c;
        assert c in s[..i];
      } else {
        assert j == i;
        assert (s[..i+1])[j] == s[i];
        assert c == ch;
        assert false;
      }
    }
  }

  i := i + 1;
```

```
  }

  assert s[..|s|] == s;
  assert forall c | c in s && 'A' <= c <= 'Z' ensures c in uppercase {
    assert c in s[..|s|];
  }
}
```

Program → GenAI

```
When using dafny resolve, the below error is emitted and resolve fails:

problem_data/p_4_3_a_extract_uppercase/program.dfy(56,4): Error: invalid UnaryExpression
   |
56 | } else {
   | ^

problem_data/p_4_3_a_extract_uppercase/program.dfy(55,6): Error: expected either 'by' or
    ↪ a semicolon following the assert expression
   |
55 | }
   | ^

problem_data/p_4_3_a_extract_uppercase/program.dfy(55,6): Error: expected either 'by' or
    ↪ a semicolon following the assert expression
   |
55 | }
   | ^

problem_data/p_4_3_a_extract_uppercase/program.dfy(72,4): Error: invalid UnaryExpression
   |
72 | }
   | ^

problem_data/p_4_3_a_extract_uppercase/program.dfy(71,6): Error: expected either 'by' or
    ↪ a semicolon following the assert expression
   |
71 | }
   | ^

problem_data/p_4_3_a_extract_uppercase/program.dfy(81,0): Error: invalid UnaryExpression
   |
81 | }
   | ^

problem_data/p_4_3_a_extract_uppercase/program.dfy(80,2): Error: expected either 'by' or
    ↪ a semicolon following the assert expression
   |
80 | }
   | ^
```

```
7 parse errors detected in program.dfy

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪  nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DmejnfMS8FrFtAxv56SsNpE2dav9D
Model: gpt-5.5-2026-04-23
Created at: 1780488711
Finish reason: stop
Usage: 1777 tokens in, and 4345 tokens out

```
{
  uppercase := "";
  var i := 0;
  while i < |s|
    invariant 0 <= i <= |s|
    invariant forall c :: c in uppercase ==> 'A' <= c && c <= 'Z'
    invariant forall c :: c in s[..i] && 'A' <= c && c <= 'Z' ==> c in uppercase
    decreases |s| - i
  {
    var ch := s[i];
    assert s[..i + 1] == s[..i] + [ch];

    if 'A' <= ch && ch <= 'Z' {
      var prev := uppercase;
      assert forall c :: c in prev ==> 'A' <= c && c <= 'Z';
      assert forall c :: c in s[..i] && 'A' <= c && c <= 'Z' ==> c in prev;

      uppercase := prev + [ch];

      forall c | c in uppercase
        ensures 'A' <= c && c <= 'Z'
      {
        if c in prev {
          assert 'A' <= c && c <= 'Z';
        } else {
          assert c in [ch];
          assert c == ch;
        }
      }

      forall c | c in s[..i + 1] && 'A' <= c && c <= 'Z'
        ensures c in uppercase
      {
        if c in s[..i] {
          assert c in prev;
          assert c in uppercase;
```

```
      } else {
        assert c in [ch];
        assert c == ch;
        assert ch in uppercase;
      }
    }
  } else {
    assert !('A' <= ch && ch <= 'Z');

    forall c | c in s[..i + 1] && 'A' <= c && c <= 'Z'
      ensures c in uppercase
    {
      if c in s[..i] {
        assert c in uppercase;
      } else {
        assert c in [ch];
        assert c == ch;
        assert 'A' <= ch && ch <= 'Z';
        assert false;
      }
    }
  }

  i := i + 1;
 }

 assert i == |s|;
 assert s[..i] == s;

 forall c | c in s && 'A' <= c && c <= 'Z'
   ensures c in uppercase
 {
   assert c in s[..i];
 }
}
```

Final Program

```
method p_4_3_a_extract_uppercase(s: string) returns (uppercase: string)
       ensures forall c :: c in uppercase ==> 'A' <= c <= 'Z'
       ensures forall c :: c in s && 'A' <= c <= 'Z' ==> c in uppercase
{
 uppercase := "";
 var i := 0;
 while i < |s|
   invariant 0 <= i <= |s|
   invariant forall c :: c in uppercase ==> 'A' <= c && c <= 'Z'
   invariant forall c :: c in s[..i] && 'A' <= c && c <= 'Z' ==> c in uppercase
   decreases |s| - i
 {
```

```
  var ch := s[i];
  assert s[..i + 1] == s[..i] + [ch];

  if 'A' <= ch && ch <= 'Z' {
    var prev := uppercase;
    assert forall c :: c in prev ==> 'A' <= c && c <= 'Z';
    assert forall c :: c in s[..i] && 'A' <= c && c <= 'Z' ==> c in prev;

    uppercase := prev + [ch];

    forall c | c in uppercase
      ensures 'A' <= c && c <= 'Z'
    {
      if c in prev {
        assert 'A' <= c && c <= 'Z';
      } else {
        assert c in [ch];
        assert c == ch;
      }
    }

    forall c | c in s[..i + 1] && 'A' <= c && c <= 'Z'
      ensures c in uppercase
    {
      if c in s[..i] {
        assert c in prev;
        assert c in uppercase;
      } else {
        assert c in [ch];
        assert c == ch;
        assert ch in uppercase;
      }
    }
  } else {
    assert !('A' <= ch && ch <= 'Z');

    forall c | c in s[..i + 1] && 'A' <= c && c <= 'Z'
      ensures c in uppercase
    {
      if c in s[..i] {
        assert c in uppercase;
      } else {
        assert c in [ch];
        assert c == ch;
        assert 'A' <= ch && ch <= 'Z';
        assert false;
      }
    }
  }

  i := i + 1;
```

```
  }

  assert i == |s|;
  assert s[..i] == s;

  forall c | c in s && 'A' <= c && c <= 'Z'
    ensures c in uppercase
  {
    assert c in s[..i];
  }
}
```

Total Token Usage

Input tokens: 2105
Output tokens: 9855
Reasoning tokens: 8312
Sum of 'total tokens': 11960

Experiment Timings

Overall Experiment started at 1780488608287, ended at 1780488792348, lasting 184061ms (184.06 seconds)
Iteration #1 started at 1780488608289, ended at 1780488711020, lasting 102731ms (102.73 seconds)
Iteration #2 started at 1780488711020, ended at 1780488792348, lasting 81328ms (81.33 seconds)

Experiment 'p_4_3_b_every_second_letter' Results

June 3, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 3
Hard fails (resolution): 2
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_4_3_b_every_second_letter
Natural language statement: Write a method that takes a string and returns every second letter of the string.
Method signature: p_4_3_b_every_second_letter(s: string) returns (result: string)

Ensures

- `|result| * 2 == |s| || |result| * 2 + 1 == |s|`
- `forall i :: 0 <= i < |result| ==> result[i] == s[2 * i]`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method that takes a string and returns every second letter of the string.

The signature should be:

p_4_3_b_every_second_letter(s: string) returns (result: string)

The method should respect the following contract:

ensures |result| * 2 == |s| || |result| * 2 + 1 == |s|, ensures forall i :: 0 <= i < |
    ↪ result| ==> result[i] == s[2 * i]

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-Dmel8o0fJualOHi208gat73Gvs5OB
Model: gpt-5.5-2026-04-23
Created at: 1780488794
Finish reason: stop
Usage: 346 tokens in, and 4627 tokens out

```
{
  result := "";
  var pos := 0;

  while pos + 1 < |s|
    invariant 0 <= pos <= |s|
    invariant |result| * 2 == pos
    invariant forall i :: 0 <= i < |result| ==> result[i] == s[2 * i]
    decreases |s| - pos
  {
    ghost var prev := result;
    ghost var oldLen := |result|;
    ghost var oldPos := pos;

    assert oldLen * 2 == oldPos;
    assert oldPos + 1 < |s|;
    assert oldPos + 2 <= |s|;
    assert |prev| == oldLen;

    result := prev + s[oldPos..oldPos + 1];
```

```
    pos := oldPos + 2;

    assert |s[oldPos..oldPos + 1]| == 1;
    assert s[oldPos..oldPos + 1][0] == s[oldPos];
    assert |result| == oldLen + 1;
    assert |result| * 2 == pos;

    assert forall i | 0 <= i < |result| ensures result[i] == s[2 * i]
    {
      if i < oldLen {
        assert result[i] == prev[i];
        assert prev[i] == s[2 * i];
      } else {
        assert i == oldLen;
        assert result[i] == s[oldPos..oldPos + 1][0];
        assert result[i] == s[oldPos];
        assert oldPos == 2 * i;
      }
    }
  }

  if pos == |s| {
    assert |result| * 2 == |s|;
  } else {
    assert pos < |s|;
    assert pos + 1 <= |s|;
    assert |s| <= pos + 1;
    assert pos + 1 == |s|;
    assert |result| * 2 + 1 == |s|;
  }
}
```

Program → GenAI

```
When using dafny resolve, the below error is emitted and resolve fails:

problem_data/p_4_3_b_every_second_letter/program.dfy(43,2): Error: invalid
    ↪ UnaryExpression
   |
43 | }
   | ^

problem_data/p_4_3_b_every_second_letter/program.dfy(42,4): Error: expected either 'by'
    ↪ or a semicolon following the assert expression
   |
42 | }
   | ^

2 parse errors detected in program.dfy
```

```
Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪  nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DmemUxcpHin54TDygEl9tmbIstiNF
Model: gpt-5.5-2026-04-23
Created at: 1780488878
Finish reason: stop
Usage: 1014 tokens in, and 2051 tokens out

```
{
  result := "";
  var pos := 0;

  while pos + 1 < |s|
    invariant 0 <= pos <= |s|
    invariant |result| * 2 == pos
    invariant forall i :: 0 <= i < |result| ==> result[i] == s[2 * i]
    decreases |s| - pos
  {
    ghost var prev := result;
    ghost var oldLen := |result|;
    ghost var oldPos := pos;

    assert oldLen * 2 == oldPos;
    assert oldPos + 1 < |s|;
    assert oldPos + 2 <= |s|;
    assert |prev| == oldLen;

    result := prev + s[oldPos..oldPos + 1];
    pos := oldPos + 2;

    assert |s[oldPos..oldPos + 1]| == 1;
    assert s[oldPos..oldPos + 1][0] == s[oldPos];
    assert |result| == oldLen + 1;
    assert |result| * 2 == pos;

    forall i | 0 <= i < |result|
      ensures result[i] == s[2 * i]
    {
      if i < oldLen {
        assert result[i] == prev[i];
        assert prev[i] == s[2 * i];
      } else {
        assert i == oldLen;
        assert result[i] == s[oldPos..oldPos + 1][0];
        assert result[i] == s[oldPos];
        assert oldPos == 2 * i;
```

```
    }
   }
  }

  if pos == |s| {
    assert |result| * 2 == |s|;
  } else {
    assert pos < |s|;
    assert pos + 1 <= |s|;
    assert |s| <= pos + 1;
    assert pos + 1 == |s|;
    assert |result| * 2 + 1 == |s|;
  }
}
```

Program → GenAI

```
When using dafny resolve, the below error is emitted and resolve fails:

problem_data/p_4_3_b_every_second_letter/program.dfy(23,14): Error: ghost variables such
    ↪ as prev are allowed only in specification contexts. prev was inferred to be ghost
    ↪ based on its declaration or initialization.
   |
23 | result := prev + s[oldPos..oldPos + 1];
   | ^^^^

problem_data/p_4_3_b_every_second_letter/program.dfy(23,23): Error: ghost variables such
    ↪ as oldPos are allowed only in specification contexts. oldPos was inferred to be
    ↪ ghost based on its declaration or initialization.
   |
23 | result := prev + s[oldPos..oldPos + 1];
   | ^^^^^^

problem_data/p_4_3_b_every_second_letter/program.dfy(23,31): Error: ghost variables such
    ↪ as oldPos are allowed only in specification contexts. oldPos was inferred to be
    ↪ ghost based on its declaration or initialization.
   |
23 | result := prev + s[oldPos..oldPos + 1];
   | ^^^^^^

problem_data/p_4_3_b_every_second_letter/program.dfy(24,11): Error: ghost variables such
    ↪ as oldPos are allowed only in specification contexts. oldPos was inferred to be
    ↪ ghost based on its declaration or initialization.
   |
24 | pos := oldPos + 2;
   | ^^^^^^

4 resolution/type errors detected in program.dfy

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
```

```
    ↪ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-Dmen8JD6SnUxl3xblpkvnskBIxN2A
Model: gpt-5.5-2026-04-23
Created at: 1780488918
Finish reason: stop
Usage: 2090 tokens in, and 1021 tokens out

```
{
  result := "";
  var pos := 0;

  while pos + 1 < |s|
    invariant 0 <= pos <= |s|
    invariant |result| * 2 == pos
    invariant forall i :: 0 <= i < |result| ==> result[i] == s[2 * i]
    decreases |s| - pos
  {
    var prev := result;
    var oldLen := |result|;
    var oldPos := pos;

    assert oldLen * 2 == oldPos;
    assert oldPos + 1 < |s|;
    assert oldPos + 2 <= |s|;
    assert |prev| == oldLen;

    result := prev + s[oldPos..oldPos + 1];
    pos := oldPos + 2;

    assert |s[oldPos..oldPos + 1]| == 1;
    assert s[oldPos..oldPos + 1][0] == s[oldPos];
    assert |result| == oldLen + 1;
    assert |result| * 2 == pos;

    forall i | 0 <= i < |result|
      ensures result[i] == s[2 * i]
    {
      if i < oldLen {
        assert i < |prev|;
        assert result[i] == prev[i];
        assert prev[i] == s[2 * i];
      } else {
        assert i == oldLen;
        assert result[i] == s[oldPos..oldPos + 1][0];
        assert result[i] == s[oldPos];
        assert oldPos == 2 * i;
      }
```

```
    }
  }

  if pos == |s| {
    assert |result| * 2 == |s|;
  } else {
    assert pos < |s|;
    assert pos + 1 <= |s|;
    assert |s| <= pos + 1;
    assert pos + 1 == |s|;
    assert |result| * 2 + 1 == |s|;
  }
}
```

Final Program

```
method p_4_3_b_every_second_letter(s: string) returns (result: string)
       ensures |result| * 2 == |s| || |result| * 2 + 1 == |s|
       ensures forall i :: 0 <= i < |result| ==> result[i] == s[2 * i]
{
  result := "";
  var pos := 0;

  while pos + 1 < |s|
    invariant 0 <= pos <= |s|
    invariant |result| * 2 == pos
    invariant forall i :: 0 <= i < |result| ==> result[i] == s[2 * i]
    decreases |s| - pos
  {
    var prev := result;
    var oldLen := |result|;
    var oldPos := pos;

    assert oldLen * 2 == oldPos;
    assert oldPos + 1 < |s|;
    assert oldPos + 2 <= |s|;
    assert |prev| == oldLen;

    result := prev + s[oldPos..oldPos + 1];
    pos := oldPos + 2;

    assert |s[oldPos..oldPos + 1]| == 1;
    assert s[oldPos..oldPos + 1][0] == s[oldPos];
    assert |result| == oldLen + 1;
    assert |result| * 2 == pos;

    forall i | 0 <= i < |result|
      ensures result[i] == s[2 * i]
    {
      if i < oldLen {
```

```
      assert i < |prev|;
      assert result[i] == prev[i];
      assert prev[i] == s[2 * i];
    } else {
      assert i == oldLen;
      assert result[i] == s[oldPos..oldPos + 1][0];
      assert result[i] == s[oldPos];
      assert oldPos == 2 * i;
    }
  }
 }

 if pos == |s| {
  assert |result| * 2 == |s|;
 } else {
  assert pos < |s|;
  assert pos + 1 <= |s|;
  assert |s| <= pos + 1;
  assert pos + 1 == |s|;
  assert |result| * 2 + 1 == |s|;
 }
}
```

Total Token Usage

Input tokens: 3450
Output tokens: 7699
Reasoning tokens: 6128
Sum of 'total tokens': 11149

Experiment Timings

Overall Experiment started at 1780488793763, ended at 1780488942356, lasting 148593ms (148.59 seconds)
Iteration #1 started at 1780488793763, ended at 1780488877629, lasting 83866ms (83.87 seconds)
Iteration #2 started at 1780488877630, ended at 1780488918309, lasting 40679ms (40.68 seconds)
Iteration #3 started at 1780488918309, ended at 1780488942356, lasting 24047ms (24.05 seconds)

Experiment 'p_4_3_c_replace_vowels' Results

June 3, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 2
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 2

Problem Specification

Problem name: p_4_3_c_replace_vowels
Natural language statement: Write a method that takes a string and returns the string, with all vowels replaced by an underscore.
Method signature: p_4_3_c_replace_vowels(s: string) returns (result: string)

Ensures

- `|result| == |s|`
- `forall i :: 0 <= i < |s| ==> (if s[i] in ['a','e','i','o','u','A','E','I','O','U'] then result[i] == '_' else result[i] == s[i])`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method that takes a string and returns the string, with all vowels replaced by an
    ↪ underscore.

The signature should be:

p_4_3_c_replace_vowels(s: string) returns (result: string)

The method should respect the following contract:

ensures |result| == |s|, ensures forall i :: 0 <= i < |s| ==> (if s[i] in ['a','e','i','o
    ↪ ','u','A','E','I','O','U'] then result[i] == '_' else result[i] == s[i])

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪ 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DmenXuJ8BxUJ4EJEvSUAETMjjoUcl
Model: gpt-5.5-2026-04-23
Created at: 1780488943
Finish reason: stop
Usage: 382 tokens in, and 1431 tokens out

```
{
  result := seq(|s|, i => if s[i] in ['a', 'e', 'i', 'o', 'u', 'A', 'E', 'I', 'O', 'U']
      ↪ then '_' else s[i]);
}
```

Program → GenAI

```
When using dafny verify, the below error is emitted and verify fails:

problem_data/p_4_3_c_replace_vowels/program.dfy(5,30): Error: index out of range
  |
5 | result := seq(|s|, i => if s[i] in ['a', 'e', 'i', 'o', 'u', 'A', 'E', 'I', 'O', 'U']
    ↪ then '_' else s[i]);
  | ^


Dafny program verifier finished with 1 verified, 1 error
```

```
Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪  nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DmeoH1djHlDP0Feu753S1wdlPzInl
Model: gpt-5.5-2026-04-23
Created at: 1780488989
Finish reason: stop
Usage: 623 tokens in, and 612 tokens out

```
{
  result := seq(|s|, i =>
    if 0 <= i < |s| && s[i] in ['a', 'e', 'i', 'o', 'u', 'A', 'E', 'I', 'O', 'U'] then '_'
        ↪  else
    if 0 <= i < |s| then s[i] else '_');
}
```

Final Program

```
method p_4_3_c_replace_vowels(s: string) returns (result: string)
       ensures |result| == |s|
       ensures forall i :: 0 <= i < |s| ==> (if s[i] in ['a','e','i','o','u','A','E','I
           ↪ ','O','U'] then result[i] == '_' else result[i] == s[i])
{
  result := seq(|s|, i =>
    if 0 <= i < |s| && s[i] in ['a', 'e', 'i', 'o', 'u', 'A', 'E', 'I', 'O', 'U'] then '_'
        ↪  else
    if 0 <= i < |s| then s[i] else '_');
}
```

Total Token Usage

Input tokens: 1005
Output tokens: 2043
Reasoning tokens: 1858
Sum of 'total tokens': 3048

Experiment Timings

Overall Experiment started at 1780488943701, ended at 1780489008861, lasting 65160ms (65.16 seconds)
Iteration #1 started at 1780488943701, ended at 1780488989275, lasting 45574ms (45.57 seconds)
Iteration #2 started at 1780488989275, ended at 1780489008861, lasting 19586ms (19.59 seconds)

Experiment 'p_4_3_d_count_vowels' Results

June 4, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_4_3_d_count_vowels
Natural language statement: Write a method that takes a string and returns the number of vowels in the string.
Method signature: p_4_3_d_count_vowels(s: string) returns (count: int)

Ensures

- `count == count_vowels(s)`

Functional Code Given

```
function count_vowels(s: string): int {
  if s == "" then 0
  else if s[0] in ['a','e','i','o','u','A','E','I','O','U'] then count_vowels(s[1..]) + 1
  else count_vowels(s[1..])
}
```

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method that takes a string and returns the number of vowels in the string.

The signature should be:

p_4_3_d_count_vowels(s: string) returns (count: int)

The method should respect the following contract:

ensures count == count_vowels(s)

The contract uses the following dafny code:

function count_vowels(s: string): int {
  if s == "" then 0
  else if s[0] in ['a','e','i','o','u','A','E','I','O','U'] then count_vowels(s[1..]) + 1
  else count_vowels(s[1..])
}

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-Dn5TH0dsw5lnRUTpRXPss3MMVoCPr
Model: gpt-5.5-2026-04-23
Created at: 1780591475
Finish reason: stop
Usage: 460 tokens in, and 631 tokens out

```
{
  if s == "" {
    count := 0;
  } else {
    var restCount := p_4_3_d_count_vowels(s[1..]);
    if s[0] in ['a','e','i','o','u','A','E','I','O','U'] {
      count := restCount + 1;
    } else {
      count := restCount;
    }
  }
```

```
}
```

Final Program

```
function count_vowels(s: string): int {
  if s == "" then 0
  else if s[0] in ['a','e','i','o','u','A','E','I','O','U'] then count_vowels(s[1..]) + 1
  else count_vowels(s[1..])
}

method p_4_3_d_count_vowels(s: string) returns (count: int)
       ensures count == count_vowels(s)
{
  if s == "" {
    count := 0;
  } else {
    var restCount := p_4_3_d_count_vowels(s[1..]);
    if s[0] in ['a','e','i','o','u','A','E','I','O','U'] {
      count := restCount + 1;
    } else {
      count := restCount;
    }
  }
}
```

Total Token Usage

Input tokens: 460
Output tokens: 631
Reasoning tokens: 512
Sum of 'total tokens': 1091

Experiment Timings

Overall Experiment started at 1780591469557, ended at 1780591545557, lasting 76000ms (76.00 seconds)
Iteration #1 started at 1780591469566, ended at 1780591545556, lasting 75990ms (75.99 seconds)

Experiment 'p_4_3_e_positions_of_vowels_array' Results

June 3, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 2
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 2

Problem Specification

Problem name: p_4_3_e_positions_of_vowels_array
Natural language statement: Write a method that takes a string and returns the positions of all vowels in the string.
Method signature: p_4_3_e_positions_of_vowels_array(s: string) returns (positions: array<int>)

Ensures

- `forall p :: p in positions[..] ==> 0 <= p < |s| && s[p] in ['a','e','i','o','u','A','E','I','O','U`
- `forall i :: 0 <= i < |s| && s[i] in ['a','e','i','o','u','A','E','I','O','U'] ==> i in positions[..]`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method that takes a string and returns the positions of all vowels in the string.

The signature should be:

p_4_3_e_positions_of_vowels_array(s: string) returns (positions: array<int>)

The method should respect the following contract:

ensures forall p :: p in positions[..] ==> 0 <= p < |s| && s[p] in ['a','e','i','o','u','
    ↪ A','E','I','O','U'], ensures forall i :: 0 <= i < |s| && s[i] in ['a','e','i','o
    ↪ ','u','A','E','I','O','U'] ==> i in positions[..]

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DmfUl047FkWr9kCMYgQDoNUXaixCE
Model: gpt-5.5-2026-04-23
Created at: 1780491623
Finish reason: stop
Usage: 436 tokens in, and 4888 tokens out

```
{
  var vowels := ['a', 'e', 'i', 'o', 'u', 'A', 'E', 'I', 'O', 'U'];
  var ps: seq<int> := [];
  var i := 0;

  while i < |s|
    invariant 0 <= i <= |s|
    invariant forall p :: p in ps ==> 0 <= p < i && s[p] in vowels
    invariant forall j :: 0 <= j < i && s[j] in vowels ==> j in ps
  {
    if s[i] in vowels {
      var newPs := ps + [i];

      forall p | p in newPs
        ensures 0 <= p < i + 1 && s[p] in vowels
      {
        if p in ps {
          assert 0 <= p < i && s[p] in vowels;
        } else {
```

```
        assert p in [i];
        assert p == i;
      }
    }

    forall j | 0 <= j < i + 1 && s[j] in vowels
      ensures j in newPs
    {
      if j < i {
        assert j in ps;
        assert j in newPs;
      } else {
        assert j == i;
        assert j in [i];
        assert j in newPs;
      }
    }

    ps := newPs;
  } else {
    forall p | p in ps
      ensures 0 <= p < i + 1 && s[p] in vowels
    {
      assert 0 <= p < i && s[p] in vowels;
    }

    forall j | 0 <= j < i + 1 && s[j] in vowels
      ensures j in ps
    {
      if j < i {
        assert j in ps;
      } else {
        assert j == i;
        assert s[j] == s[i];
        assert false;
      }
    }
  }

  i := i + 1;
}

assert i == |s|;
positions := new int[|ps|](k => ps[k]);

assert positions.Length == |ps|;
forall k | 0 <= k < |ps|
  ensures positions[..][k] == ps[k]
{
  assert positions[k] == ps[k];
  assert positions[..][k] == positions[k];
```

```
  }
  assert |positions[..]| == |ps|;
  assert positions[..] == ps;

  assert vowels == ['a', 'e', 'i', 'o', 'u', 'A', 'E', 'I', 'O', 'U'];
  assert forall p :: p in positions[..] ==> 0 <= p < |s| && s[p] in ['a', 'e', 'i', 'o',
      ↪ 'u', 'A', 'E', 'I', 'O', 'U'];
  assert forall j :: 0 <= j < |s| && s[j] in ['a', 'e', 'i', 'o', 'u', 'A', 'E', 'I', 'O
      ↪ ', 'U'] ==> j in positions[..];
}
```

Program → GenAI

```
When using dafny verify, the below error is emitted and verify fails:

problem_data/p_4_3_e_positions_of_vowels_array/program.dfy(66,36): Error: index out of
    ↪ range
   |
66 | positions := new int[|ps|](k => ps[k]);
   |                            ^


Dafny program verifier finished with 1 verified, 1 error

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪  nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DmfWHCXqwl6B8KsoZTg36Mp3CPR3a
Model: gpt-5.5-2026-04-23
Created at: 1780491717
Finish reason: stop
Usage: 1259 tokens in, and 1795 tokens out

```
{
  var vowels := ['a', 'e', 'i', 'o', 'u', 'A', 'E', 'I', 'O', 'U'];
  var ps: seq<int> := [];
  var i := 0;

  while i < |s|
    invariant 0 <= i <= |s|
    invariant forall p :: p in ps ==> 0 <= p < i && s[p] in vowels
    invariant forall j :: 0 <= j < i && s[j] in vowels ==> j in ps
  {
    if s[i] in vowels {
      var newPs := ps + [i];

      forall p | p in newPs
```

```
      ensures 0 <= p < i + 1 && s[p] in vowels
    {
      if p in ps {
        assert 0 <= p < i && s[p] in vowels;
      } else {
        assert p in [i];
        assert p == i;
      }
    }

    forall j | 0 <= j < i + 1 && s[j] in vowels
      ensures j in newPs
    {
      if j < i {
        assert j in ps;
        assert j in newPs;
      } else {
        assert j == i;
        assert j in [i];
        assert j in newPs;
      }
    }

    ps := newPs;
  } else {
    forall p | p in ps
      ensures 0 <= p < i + 1 && s[p] in vowels
    {
      assert 0 <= p < i && s[p] in vowels;
    }

    forall j | 0 <= j < i + 1 && s[j] in vowels
      ensures j in ps
    {
      if j < i {
        assert j in ps;
      } else {
        assert j == i;
        assert s[j] == s[i];
        assert false;
      }
    }
  }

  i := i + 1;
}

assert i == |s|;
positions := new int[|ps|](k => if 0 <= k && k < |ps| then ps[k] else 0);

assert positions.Length == |ps|;
```

```
  forall k | 0 <= k < |ps|
    ensures positions[..][k] == ps[k]
  {
    assert positions[k] == ps[k];
    assert positions[..][k] == positions[k];
  }
  assert |positions[..]| == |ps|;
  assert positions[..] == ps;

  assert vowels == ['a', 'e', 'i', 'o', 'u', 'A', 'E', 'I', 'O', 'U'];
  assert forall p :: p in positions[..] ==> 0 <= p < |s| && s[p] in ['a', 'e', 'i', 'o',
      ↪ 'u', 'A', 'E', 'I', 'O', 'U'];
  assert forall j :: 0 <= j < |s| && s[j] in ['a', 'e', 'i', 'o', 'u', 'A', 'E', 'I', 'O
      ↪ ', 'U'] ==> j in positions[..];
}
```

Final Program

```
method p_4_3_e_positions_of_vowels_array(s: string) returns (positions: array<int>)
       ensures forall p :: p in positions[..] ==> 0 <= p < |s| && s[p] in ['a','e','i','o
           ↪ ','u','A','E','I','O','U']
       ensures forall i :: 0 <= i < |s| && s[i] in ['a','e','i','o','u','A','E','I','O','
           ↪ U'] ==> i in positions[..]
{
  var vowels := ['a', 'e', 'i', 'o', 'u', 'A', 'E', 'I', 'O', 'U'];
  var ps: seq<int> := [];
  var i := 0;

  while i < |s|
    invariant 0 <= i <= |s|
    invariant forall p :: p in ps ==> 0 <= p < i && s[p] in vowels
    invariant forall j :: 0 <= j < i && s[j] in vowels ==> j in ps
  {
    if s[i] in vowels {
      var newPs := ps + [i];

      forall p | p in newPs
        ensures 0 <= p < i + 1 && s[p] in vowels
      {
        if p in ps {
          assert 0 <= p < i && s[p] in vowels;
        } else {
          assert p in [i];
          assert p == i;
        }
      }

      forall j | 0 <= j < i + 1 && s[j] in vowels
        ensures j in newPs
      {
```

```
      if j < i {
        assert j in ps;
        assert j in newPs;
      } else {
        assert j == i;
        assert j in [i];
        assert j in newPs;
      }
    }

    ps := newPs;
  } else {
    forall p | p in ps
      ensures 0 <= p < i + 1 && s[p] in vowels
    {
      assert 0 <= p < i && s[p] in vowels;
    }

    forall j | 0 <= j < i + 1 && s[j] in vowels
      ensures j in ps
    {
      if j < i {
        assert j in ps;
      } else {
        assert j == i;
        assert s[j] == s[i];
        assert false;
      }
    }
  }

  i := i + 1;
}

assert i == |s|;
positions := new int[|ps|](k => if 0 <= k && k < |ps| then ps[k] else 0);

assert positions.Length == |ps|;
forall k | 0 <= k < |ps|
  ensures positions[..][k] == ps[k]
{
  assert positions[k] == ps[k];
  assert positions[..][k] == positions[k];
}
assert |positions[..]| == |ps|;
assert positions[..] == ps;

assert vowels == ['a', 'e', 'i', 'o', 'u', 'A', 'E', 'I', 'O', 'U'];
assert forall p :: p in positions[..] ==> 0 <= p < |s| && s[p] in ['a', 'e', 'i', 'o',
    ↪ 'u', 'A', 'E', 'I', 'O', 'U'];
```

```
  assert forall j :: 0 <= j < |s| && s[j] in ['a', 'e', 'i', 'o', 'u', 'A', 'E', 'I', 'O
      ↪ ', 'U'] ==> j in positions[..];
}
```

Total Token Usage

Input tokens: 1695
Output tokens: 6683
Reasoning tokens: 5120
Sum of 'total tokens': 8378

Experiment Timings

Overall Experiment started at 1780491622888, ended at 1780491753105, lasting 130217ms (130.22 seconds)
Iteration #1 started at 1780491622888, ended at 1780491716910, lasting 94022ms (94.02 seconds)
Iteration #2 started at 1780491716911, ended at 1780491753105, lasting 36194ms (36.19 seconds)

Experiment 'p_4_3_e_positions_of_vowels_seq' Results

June 3, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume :` 0
Responses containing `expect :` 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_4_3_e_positions_of_vowels_seq
Natural language statement: Write a method that takes a string and returns the positions of all vowels in the string.
Method signature: p_4_3_e_positions_of_vowels_seq(s: string) returns (positions: seq<int>)

Ensures

- `forall p :: p in positions ==> 0 <= p < |s| && s[p] in ['a','e','i','o','u','A','E','I','O','U']`
- `forall i :: 0 <= i < |s| && s[i] in ['a','e','i','o','u','A','E','I','O','U'] ==> i in positions`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method that takes a string and returns the positions of all vowels in the string.

The signature should be:

p_4_3_e_positions_of_vowels_seq(s: string) returns (positions: seq<int>)

The method should respect the following contract:

ensures forall p :: p in positions ==> 0 <= p < |s| && s[p] in ['a','e','i','o','u','A','
    ↪ E','I','O','U'], ensures forall i :: 0 <= i < |s| && s[i] in ['a','e','i','o','u
    ↪ ','A','E','I','O','U'] ==> i in positions

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DmfTzspI62mS2NvV6pYE4i3AVDGGU
Model: gpt-5.5-2026-04-23
Created at: 1780491575
Finish reason: stop
Usage: 430 tokens in, and 1743 tokens out

```
{
  positions := [];
  var i := 0;
  while i < |s|
    invariant 0 <= i <= |s|
    invariant forall p :: p in positions ==> 0 <= p < i && s[p] in ['a','e','i','o','u','A
        ↪ ','E','I','O','U']
    invariant forall j :: 0 <= j < i && s[j] in ['a','e','i','o','u','A','E','I','O','U']
        ↪ ==> j in positions
  {
    if s[i] in ['a','e','i','o','u','A','E','I','O','U'] {
      positions := positions + [i];
    }
    i := i + 1;
  }
}
```

Final Program

```
method p_4_3_e_positions_of_vowels_seq(s: string) returns (positions: seq<int>)
       ensures forall p :: p in positions ==> 0 <= p < |s| && s[p] in ['a','e','i','o','u
           ↪ ','A','E','I','O','U']
       ensures forall i :: 0 <= i < |s| && s[i] in ['a','e','i','o','u','A','E','I','O','
           ↪ U'] ==> i in positions
{
  positions := [];
  var i := 0;
  while i < |s|
    invariant 0 <= i <= |s|
    invariant forall p :: p in positions ==> 0 <= p < i && s[p] in ['a','e','i','o','u','A
        ↪ ','E','I','O','U']
    invariant forall j :: 0 <= j < i && s[j] in ['a','e','i','o','u','A','E','I','O','U']
        ↪ ==> j in positions
  {
    if s[i] in ['a','e','i','o','u','A','E','I','O','U'] {
      positions := positions + [i];
    }
    i := i + 1;
  }
}
```

Total Token Usage

Input tokens: 430
Output tokens: 1743
Reasoning tokens: 1536
Sum of 'total tokens': 2173

Experiment Timings

Overall Experiment started at 1780491575261, ended at 1780491622142, lasting 46881ms (46.88 seconds)
Iteration #1 started at 1780491575262, ended at 1780491622142, lasting 46880ms (46.88 seconds)

Experiment 'p_4_4_month_with_highest_temperature_array' Results

June 3, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_4_4_month_with_highest_temperature_array
Natural language statement: Write a method that reads a sequence of twelve temperature values and returns the number of the month with the highest temperature.
Method signature: p_4_4_month_with_highest_temperature_array(inputs: array<real>) returns (hottest: int)

Ensures

- `1 <= hottest <= 12`
- `forall i :: 0 <= i <= inputs.Length - 1 ==> inputs[hottest - 1] >= inputs[i]`

Requires

- `inputs.Length == 12`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method that reads a sequence of twelve temperature values and returns the number
    ↪ of the month with the highest temperature.

The signature should be:

p_4_4_month_with_highest_temperature_array(inputs: array<real>) returns (hottest: int)

The method should respect the following contract:

requires inputs.Length == 12, ensures 1 <= hottest <= 12, ensures forall i :: 0 <= i <=
    ↪ inputs.Length - 1 ==> inputs[hottest - 1] >= inputs[i]

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DmfXBEkQvr0YD0tgd5fnCW5eVkt2k
Model: gpt-5.5-2026-04-23
Created at: 1780491773
Finish reason: stop
Usage: 354 tokens in, and 646 tokens out

```
{
  hottest := 1;
  var idx := 1;
  while idx < inputs.Length
    invariant 1 <= idx <= inputs.Length
    invariant 1 <= hottest <= idx
    invariant forall i :: 0 <= i < idx ==> inputs[hottest - 1] >= inputs[i]
    decreases inputs.Length - idx
  {
    if inputs[idx] > inputs[hottest - 1] {
      hottest := idx + 1;
    }
    idx := idx + 1;
  }
}
```

Final Program

```
method p_4_4_month_with_highest_temperature_array(inputs: array<real>) returns (hottest:
    ↪ int)
       requires inputs.Length == 12
       ensures 1 <= hottest <= 12
       ensures forall i :: 0 <= i <= inputs.Length - 1 ==> inputs[hottest - 1] >= inputs[
           ↪ i]
{
  hottest := 1;
  var idx := 1;
  while idx < inputs.Length
    invariant 1 <= idx <= inputs.Length
    invariant 1 <= hottest <= idx
    invariant forall i :: 0 <= i < idx ==> inputs[hottest - 1] >= inputs[i]
    decreases inputs.Length - idx
  {
    if inputs[idx] > inputs[hottest - 1] {
      hottest := idx + 1;
    }
    idx := idx + 1;
  }
}
```

Total Token Usage

Input tokens: 354
Output tokens: 646
Reasoning tokens: 512
Sum of 'total tokens': 1000

Experiment Timings

Overall Experiment started at 1780491773517, ended at 1780491793124, lasting 19607ms (19.61 seconds)
Iteration #1 started at 1780491773517, ended at 1780491793124, lasting 19607ms (19.61 seconds)

Experiment 'p_4_4_month_with_highest_temperature_seq' Results

June 3, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_4_4_month_with_highest_temperature_seq
Natural language statement: Write a method that reads a sequence of twelve temperature values and returns the number of the month with the highest temperature.
Method signature: p_4_4_month_with_highest_temperature_seq(inputs: seq<real>) returns (hottest: int)

Ensures

- `1 <= hottest <= 12`
- `forall i :: 0 <= i <= |inputs| - 1 ==> inputs[hottest - 1] >= inputs[i]`

Requires

- `|inputs| == 12`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method that reads a sequence of twelve temperature values and returns the number
    ↪ of the month with the highest temperature.

The signature should be:

p_4_4_month_with_highest_temperature_seq(inputs: seq<real>) returns (hottest: int)

The method should respect the following contract:

requires |inputs| == 12, ensures 1 <= hottest <= 12, ensures forall i :: 0 <= i <= |
    ↪ inputs| - 1 ==> inputs[hottest - 1] >= inputs[i]

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DmfWsvOYrs1eUUDGyZbTmY0Dv2Gsr
Model: gpt-5.5-2026-04-23
Created at: 1780491754
Finish reason: stop
Usage: 358 tokens in, and 654 tokens out

```
{
  var maxIdx := 0;
  var i := 1;
  while i < |inputs|
    invariant 1 <= i <= |inputs|
    invariant 0 <= maxIdx < i
    invariant forall j :: 0 <= j < i ==> inputs[maxIdx] >= inputs[j]
    decreases |inputs| - i
  {
    if inputs[i] > inputs[maxIdx] {
      maxIdx := i;
    }
    i := i + 1;
  }
  hottest := maxIdx + 1;
}
```

Final Program

```
method p_4_4_month_with_highest_temperature_seq(inputs: seq<real>) returns (hottest: int)
       requires |inputs| == 12
       ensures 1 <= hottest <= 12
       ensures forall i :: 0 <= i <= |inputs| - 1 ==> inputs[hottest - 1] >= inputs[i]
{
  var maxIdx := 0;
  var i := 1;
  while i < |inputs|
    invariant 1 <= i <= |inputs|
    invariant 0 <= maxIdx < i
    invariant forall j :: 0 <= j < i ==> inputs[maxIdx] >= inputs[j]
    decreases |inputs| - i
  {
    if inputs[i] > inputs[maxIdx] {
      maxIdx := i;
    }
    i := i + 1;
  }
  hottest := maxIdx + 1;
}
```

Total Token Usage

Input tokens: 358
Output tokens: 654
Reasoning tokens: 512
Sum of 'total tokens': 1012

Experiment Timings

Overall Experiment started at 1780491754217, ended at 1780491772800, lasting 18583ms (18.58 seconds)
Iteration #1 started at 1780491754218, ended at 1780491772800, lasting 18582ms (18.58 seconds)

Experiment 'p_4_5_analyse_floats_array' Results

June 3, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_4_5_analyse_floats_array
Natural language statement: Write a method that takes a sequence of floatingpoint numbers and returns: - the average of the values. - the smallest of the values. - the largest of the values. - the range, that is the difference between the smallest and largest.
Method signature: p_4_5_analyse_floats_array(values: array<real>) returns (average: real, min: real, max: real, range: real)

Ensures

- `average == (sum(values[..]) / values.Length as real)`
- `forall v :: v in values[..] ==> min <= v`
- `forall v :: v in values[..] ==> max >= v`
- `range == max - min`
- `exists v :: v in values[..] && min == v`
- `exists v :: v in values[..] && max == v`

Requires

- `0 < values.Length`

Functional Code Given

```
function sum(values: seq<real>): real
{
  if |values| == 0 then 0.0
  else values[0] + sum(values[1..])
}
```

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method that takes a sequence of floatingpoint numbers and returns: - the average
    ↪ of the values. - the smallest of the values. - the largest of the values. - the
    ↪ range, that is the difference between the smallest and largest.

The signature should be:

p_4_5_analyse_floats_array(values: array<real>) returns (average: real, min: real, max:
    ↪ real, range: real)

The method should respect the following contract:

requires 0 < values.Length, ensures average == (sum(values[..]) / values.Length as real),
    ↪  ensures forall v :: v in values[..] ==> min <= v, ensures forall v :: v in values
    ↪ [..] ==> max >= v, ensures range == max - min, ensures exists v :: v in values[..]
    ↪  && min == v, ensures exists v :: v in values[..] && max == v

The contract uses the following dafny code:

function sum(values: seq<real>): real
{
  if |values| == 0 then 0.0
  else values[0] + sum(values[1..])
}

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DmfbpGRC3dbqmkIgMtUyDQluGieKN
Model: gpt-5.5-2026-04-23
Created at: 1780492061
Finish reason: stop
Usage: 638 tokens in, and 4461 tokens out

```
{
  var i := values.Length - 1;
  var total := values[i];
  min := values[i];
  max := values[i];
```

```
assert values[i..] == [values[i]];
assert sum(values[i..]) == values[i];
assert forall v :: v in values[i..] ==> min <= v;
assert forall v :: v in values[i..] ==> max >= v;
assert min in values[i..];
assert max in values[i..];

while 0 < i
  invariant 0 <= i < values.Length
  invariant total == sum(values[i..])
  invariant forall v :: v in values[i..] ==> min <= v
  invariant forall v :: v in values[i..] ==> max >= v
  invariant min in values[i..]
  invariant max in values[i..]
  decreases i
{
  var oldI := i;
  var oldMin := min;
  var oldMax := max;

  i := i - 1;
  var x := values[i];

  assert oldI == i + 1;
  assert values[i..] == [x] + values[oldI..];
  assert values[i..][0] == x;
  assert values[i..][1..] == values[oldI..];

  total := x + total;
  assert total == x + sum(values[oldI..]);
  assert sum(values[i..]) == x + sum(values[i..][1..]);
  assert sum(values[i..]) == total;

  if x < oldMin {
    min := x;
    assert min in values[i..];
    forall v | v in values[i..]
      ensures min <= v
    {
      if v == x {
      } else {
        assert v in values[oldI..];
        assert oldMin <= v;
        assert min < oldMin;
      }
    }
  } else {
    min := oldMin;
    assert min <= x;
    assert min in values[oldI..];
```

```
      assert min in values[i..];
      forall v | v in values[i..]
        ensures min <= v
      {
        if v == x {
        } else {
          assert v in values[oldI..];
          assert oldMin <= v;
        }
      }
    }

    if x > oldMax {
      max := x;
      assert max in values[i..];
      forall v | v in values[i..]
        ensures max >= v
      {
        if v == x {
        } else {
          assert v in values[oldI..];
          assert oldMax >= v;
          assert max > oldMax;
        }
      }
    } else {
      max := oldMax;
      assert max >= x;
      assert max in values[oldI..];
      assert max in values[i..];
      forall v | v in values[i..]
        ensures max >= v
      {
        if v == x {
        } else {
          assert v in values[oldI..];
          assert oldMax >= v;
        }
      }
    }
  }

  assert i == 0;
  assert values[i..] == values[..];
  assert total == sum(values[..]);
  assert min in values[..];
  assert max in values[..];

  average := total / (values.Length as real);
  range := max - min;
}
```

Final Program

```
function sum(values: seq<real>): real
{
  if |values| == 0 then 0.0
  else values[0] + sum(values[1..])
}

method p_4_5_analyse_floats_array(values: array<real>) returns (average: real, min: real,
    ↪  max: real, range: real)
       requires 0 < values.Length
       ensures average == (sum(values[..]) / values.Length as real)
       ensures forall v :: v in values[..] ==> min <= v
       ensures forall v :: v in values[..] ==> max >= v
       ensures range == max - min
       ensures exists v :: v in values[..] && min == v
       ensures exists v :: v in values[..] && max == v
{
  var i := values.Length - 1;
  var total := values[i];
  min := values[i];
  max := values[i];

  assert values[i..] == [values[i]];
  assert sum(values[i..]) == values[i];
  assert forall v :: v in values[i..] ==> min <= v;
  assert forall v :: v in values[i..] ==> max >= v;
  assert min in values[i..];
  assert max in values[i..];

  while 0 < i
    invariant 0 <= i < values.Length
    invariant total == sum(values[i..])
    invariant forall v :: v in values[i..] ==> min <= v
    invariant forall v :: v in values[i..] ==> max >= v
    invariant min in values[i..]
    invariant max in values[i..]
    decreases i
  {
    var oldI := i;
    var oldMin := min;
    var oldMax := max;

    i := i - 1;
    var x := values[i];

    assert oldI == i + 1;
    assert values[i..] == [x] + values[oldI..];
    assert values[i..][0] == x;
    assert values[i..][1..] == values[oldI..];

    total := x + total;
```

```
assert total == x + sum(values[oldI..]);
assert sum(values[i..]) == x + sum(values[i..][1..]);
assert sum(values[i..]) == total;

if x < oldMin {
  min := x;
  assert min in values[i..];
  forall v | v in values[i..]
    ensures min <= v
  {
    if v == x {
    } else {
      assert v in values[oldI..];
      assert oldMin <= v;
      assert min < oldMin;
    }
  }
} else {
  min := oldMin;
  assert min <= x;
  assert min in values[oldI..];
  assert min in values[i..];
  forall v | v in values[i..]
    ensures min <= v
  {
    if v == x {
    } else {
      assert v in values[oldI..];
      assert oldMin <= v;
    }
  }
}

if x > oldMax {
  max := x;
  assert max in values[i..];
  forall v | v in values[i..]
    ensures max >= v
  {
    if v == x {
    } else {
      assert v in values[oldI..];
      assert oldMax >= v;
      assert max > oldMax;
    }
  }
} else {
  max := oldMax;
  assert max >= x;
  assert max in values[oldI..];
  assert max in values[i..];
```

```
    forall v | v in values[i..]
      ensures max >= v
    {
      if v == x {
      } else {
        assert v in values[oldI..];
        assert oldMax >= v;
      }
    }
  }
 }

 assert i == 0;
 assert values[i..] == values[..];
 assert total == sum(values[..]);
 assert min in values[..];
 assert max in values[..];

 average := total / (values.Length as real);
 range := max - min;
}
```

Total Token Usage

Input tokens: 638
Output tokens: 4461
Reasoning tokens: 3584
Sum of 'total tokens': 5099

Experiment Timings

Overall Experiment started at 1780492061762, ended at 1780492149436, lasting 87674ms (87.67 seconds)
Iteration #1 started at 1780492061762, ended at 1780492149436, lasting 87674ms (87.67 seconds)

Experiment 'p_4_5_analyse_floats_seq' Results

June 3, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 2
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 2

Problem Specification

Problem name: p_4_5_analyse_floats_seq
Natural language statement: Write a method that takes a sequence of floatingpoint numbers and returns: - the average of the values. - the smallest of the values. - the largest of the values. - the range, that is the difference between the smallest and largest.
Method signature: p_4_5_analyse_floats_seq(values: seq<real>) returns (average: real, min: real, max: real, range: real)

Ensures

- `average == (sum(values) / |values| as real)`
- `forall v :: v in values ==> min <= v`
- `forall v :: v in values ==> max >= v`
- `range == max - min`
- `exists v :: v in values && min == v`
- `exists v :: v in values && max == v`

Requires

- `0 < |values|`

Functional Code Given

```
function sum(values: seq<real>): real
{
  if |values| == 0 then 0.0
  else values[0] + sum(values[1..])
}
```

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method that takes a sequence of floatingpoint numbers and returns: - the average
    ↪ of the values. - the smallest of the values. - the largest of the values. - the
    ↪ range, that is the difference between the smallest and largest.

The signature should be:

p_4_5_analyse_floats_seq(values: seq<real>) returns (average: real, min: real, max: real,
    ↪  range: real)

The method should respect the following contract:

requires 0 < |values|, ensures average == (sum(values) / |values| as real), ensures
    ↪ forall v :: v in values ==> min <= v, ensures forall v :: v in values ==> max >= v
    ↪ , ensures range == max - min, ensures exists v :: v in values && min == v, ensures
    ↪  exists v :: v in values && max == v

The contract uses the following dafny code:

function sum(values: seq<real>): real
{
  if |values| == 0 then 0.0
  else values[0] + sum(values[1..])
}

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DmfXWRJi8rVfUEisnqGXaT038NUd4
Model: gpt-5.5-2026-04-23
Created at: 1780491794
Finish reason: stop
Usage: 624 tokens in, and 9146 tokens out

```
{
  if |values| == 1 {
    average := values[0];
    min := values[0];
    max := values[0];
```

```
  range := max - min;

  assert values == [values[0]];
  assert values[0] in values;

  forall v | v in values
    ensures min <= v
  {
    assert v == values[0];
  }

  forall v | v in values
    ensures max >= v
  {
    assert v == values[0];
  }

  assert exists v :: v in values && min == v;
  assert exists v :: v in values && max == v;
} else {
  assert 1 < |values|;
  var tail := values[1..];
  assert 0 < |tail|;
  assert |values| == |tail| + 1;
  assert values == [values[0]] + tail;
  assert forall v :: (v in values) <==> (v == values[0] || v in tail);

  var tailAverage, tailMin, tailMax, tailRange := p_4_5_analyse_floats_seq(tail);
  assert tailRange == tailMax - tailMin;

  var tailLength := |tail| as real;
  assert tailLength != 0.0;
  assert (|values| as real) == tailLength + 1.0;

  average := (values[0] + tailAverage * tailLength) / (|values| as real);

  if values[0] <= tailMin {
    min := values[0];
    assert values[0] in values;
    assert exists v :: v in values && min == v;
  } else {
    min := tailMin;
    ghost var w: real :| w in tail && tailMin == w;
    assert w in values;
    assert exists v :: v in values && min == v;
  }

  if values[0] >= tailMax {
    max := values[0];
    assert values[0] in values;
    assert exists v :: v in values && max == v;
```

```
} else {
  max := tailMax;
  ghost var w: real :| w in tail && tailMax == w;
  assert w in values;
  assert exists v :: v in values && max == v;
}

forall v | v in values
  ensures min <= v
{
  if v == values[0] {
    if values[0] <= tailMin {
      assert min == values[0];
    } else {
      assert min == tailMin;
      assert tailMin < values[0];
    }
  } else {
    assert v in tail;
    assert tailMin <= v;
    if values[0] <= tailMin {
      assert min == values[0];
      assert values[0] <= v;
    } else {
      assert min == tailMin;
    }
  }
}

forall v | v in values
  ensures max >= v
{
  if v == values[0] {
    if values[0] >= tailMax {
      assert max == values[0];
    } else {
      assert max == tailMax;
      assert tailMax > values[0];
    }
  } else {
    assert v in tail;
    assert tailMax >= v;
    if values[0] >= tailMax {
      assert max == values[0];
      assert values[0] >= v;
    } else {
      assert max == tailMax;
    }
  }
}
```

```
    range := max - min;
  }
}
```

Program → GenAI

```
When using dafny verify, the below error is emitted and verify fails:

problem_data/p_4_5_analyse_floats_seq/program.dfy(15,0): Error: a postcondition could not
    ↪  be proved on this return path
   |
15 | {
   | ^

problem_data/p_4_5_analyse_floats_seq/program.dfy(9,17): Related location: this is the
    ↪ postcondition that could not be proved
  |
9 |  ensures average == (sum(values) / |values| as real)
  |  ^^


Dafny program verifier finished with 2 verified, 1 error

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪  nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DmfaOLkiRTAZvF8dgtNorfoAqYN5I
Model: gpt-5.5-2026-04-23
Created at: 1780491972
Finish reason: stop
Usage: 1594 tokens in, and 4558 tokens out

```
{
  if |values| == 1 {
    average := values[0];
    min := values[0];
    max := values[0];
    range := max - min;

    assert values == [values[0]];
    assert values[1..] == [];
    assert sum(values[1..]) == 0.0;
    assert sum(values) == values[0] + sum(values[1..]);
    assert sum(values) == values[0];
    assert (|values| as real) == 1.0;
    assert average == sum(values) / (|values| as real);
```

```
  assert values[0] in values;

  forall v | v in values
    ensures min <= v
  {
    assert v == values[0];
  }

  forall v | v in values
    ensures max >= v
  {
    assert v == values[0];
  }

  assert exists v :: v in values && min == v;
  assert exists v :: v in values && max == v;
} else {
  assert 1 < |values|;
  var tail := values[1..];
  assert 0 < |tail|;
  assert |values| == |tail| + 1;
  assert values == [values[0]] + tail;
  assert forall v :: (v in values) <==> (v == values[0] || v in tail);

  var tailAverage, tailMin, tailMax, tailRange := p_4_5_analyse_floats_seq(tail);
  assert tailAverage == sum(tail) / (|tail| as real);
  assert tailRange == tailMax - tailMin;

  var tailLength := |tail| as real;
  assert tailLength == (|tail| as real);
  assert tailLength != 0.0;
  assert (|values| as real) == tailLength + 1.0;

  average := (values[0] + tailAverage * tailLength) / (|values| as real);

  calc {
    tailAverage * tailLength;
    == {
      assert tailAverage == sum(tail) / tailLength;
    }
    (sum(tail) / tailLength) * tailLength;
    == {
      assert tailLength != 0.0;
    }
    sum(tail);
  }

  assert values[1..] == tail;
  assert sum(values) == values[0] + sum(values[1..]);
  assert sum(values) == values[0] + sum(tail);
```

```
calc {
  average;
  == {
  }
  (values[0] + tailAverage * tailLength) / (|values| as real);
  == {
    assert tailAverage * tailLength == sum(tail);
  }
  (values[0] + sum(tail)) / (|values| as real);
  == {
    assert sum(values) == values[0] + sum(tail);
  }
  sum(values) / (|values| as real);
}

if values[0] <= tailMin {
  min := values[0];
  assert values[0] in values;
  assert exists v :: v in values && min == v;
} else {
  min := tailMin;
  ghost var w: real :| w in tail && tailMin == w;
  assert w in values;
  assert exists v :: v in values && min == v;
}

if values[0] >= tailMax {
  max := values[0];
  assert values[0] in values;
  assert exists v :: v in values && max == v;
} else {
  max := tailMax;
  ghost var w: real :| w in tail && tailMax == w;
  assert w in values;
  assert exists v :: v in values && max == v;
}

forall v | v in values
  ensures min <= v
{
  if v == values[0] {
    if values[0] <= tailMin {
      assert min == values[0];
    } else {
      assert min == tailMin;
      assert tailMin < values[0];
    }
  } else {
    assert v in tail;
    assert tailMin <= v;
    if values[0] <= tailMin {
```

```
        assert min == values[0];
        assert values[0] <= v;
      } else {
        assert min == tailMin;
      }
    }
  }

  forall v | v in values
    ensures max >= v
  {
    if v == values[0] {
      if values[0] >= tailMax {
        assert max == values[0];
      } else {
        assert max == tailMax;
        assert tailMax > values[0];
      }
    } else {
      assert v in tail;
      assert tailMax >= v;
      if values[0] >= tailMax {
        assert max == values[0];
        assert values[0] >= v;
      } else {
        assert max == tailMax;
      }
    }
  }

  range := max - min;
 }
}
```

Final Program

```
function sum(values: seq<real>): real
{
 if |values| == 0 then 0.0
 else values[0] + sum(values[1..])
}

method p_4_5_analyse_floats_seq(values: seq<real>) returns (average: real, min: real, max
    ↪ : real, range: real)
      requires 0 < |values|
      ensures average == (sum(values) / |values| as real)
      ensures forall v :: v in values ==> min <= v
      ensures forall v :: v in values ==> max >= v
      ensures range == max - min
      ensures exists v :: v in values && min == v
```

```
      ensures exists v :: v in values && max == v
{
  if |values| == 1 {
    average := values[0];
    min := values[0];
    max := values[0];
    range := max - min;

    assert values == [values[0]];
    assert values[1..] == [];
    assert sum(values[1..]) == 0.0;
    assert sum(values) == values[0] + sum(values[1..]);
    assert sum(values) == values[0];
    assert (|values| as real) == 1.0;
    assert average == sum(values) / (|values| as real);

    assert values[0] in values;

    forall v | v in values
      ensures min <= v
    {
      assert v == values[0];
    }

    forall v | v in values
      ensures max >= v
    {
      assert v == values[0];
    }

    assert exists v :: v in values && min == v;
    assert exists v :: v in values && max == v;
  } else {
    assert 1 < |values|;
    var tail := values[1..];
    assert 0 < |tail|;
    assert |values| == |tail| + 1;
    assert values == [values[0]] + tail;
    assert forall v :: (v in values) <==> (v == values[0] || v in tail);

    var tailAverage, tailMin, tailMax, tailRange := p_4_5_analyse_floats_seq(tail);
    assert tailAverage == sum(tail) / (|tail| as real);
    assert tailRange == tailMax - tailMin;

    var tailLength := |tail| as real;
    assert tailLength == (|tail| as real);
    assert tailLength != 0.0;
    assert (|values| as real) == tailLength + 1.0;

    average := (values[0] + tailAverage * tailLength) / (|values| as real);
```

```
calc {
  tailAverage * tailLength;
  == {
    assert tailAverage == sum(tail) / tailLength;
  }
  (sum(tail) / tailLength) * tailLength;
  == {
    assert tailLength != 0.0;
  }
  sum(tail);
}

assert values[1..] == tail;
assert sum(values) == values[0] + sum(values[1..]);
assert sum(values) == values[0] + sum(tail);

calc {
  average;
  == {
  }
  (values[0] + tailAverage * tailLength) / (|values| as real);
  == {
    assert tailAverage * tailLength == sum(tail);
  }
  (values[0] + sum(tail)) / (|values| as real);
  == {
    assert sum(values) == values[0] + sum(tail);
  }
  sum(values) / (|values| as real);
}

if values[0] <= tailMin {
  min := values[0];
  assert values[0] in values;
  assert exists v :: v in values && min == v;
} else {
  min := tailMin;
  ghost var w: real :| w in tail && tailMin == w;
  assert w in values;
  assert exists v :: v in values && min == v;
}

if values[0] >= tailMax {
  max := values[0];
  assert values[0] in values;
  assert exists v :: v in values && max == v;
} else {
  max := tailMax;
  ghost var w: real :| w in tail && tailMax == w;
  assert w in values;
  assert exists v :: v in values && max == v;
```

```
    }

    forall v | v in values
      ensures min <= v
    {
      if v == values[0] {
        if values[0] <= tailMin {
          assert min == values[0];
        } else {
          assert min == tailMin;
          assert tailMin < values[0];
        }
      } else {
        assert v in tail;
        assert tailMin <= v;
        if values[0] <= tailMin {
          assert min == values[0];
          assert values[0] <= v;
        } else {
          assert min == tailMin;
        }
      }
    }

    forall v | v in values
      ensures max >= v
    {
      if v == values[0] {
        if values[0] >= tailMax {
          assert max == values[0];
        } else {
          assert max == tailMax;
          assert tailMax > values[0];
        }
      } else {
        assert v in tail;
        assert tailMax >= v;
        if values[0] >= tailMax {
          assert max == values[0];
          assert values[0] >= v;
        } else {
          assert max == tailMax;
        }
      }
    }

    range := max - min;
  }
}
```

Total Token Usage

Input tokens: 2218
Output tokens: 13704
Reasoning tokens: 11445
Sum of 'total tokens': 15922

Experiment Timings

Overall Experiment started at 1780491793847, ended at 1780492060642, lasting 266795ms (266.80 seconds)
Iteration #1 started at 1780491793847, ended at 1780491972601, lasting 178754ms (178.75 seconds)
Iteration #2 started at 1780491972602, ended at 1780492060642, lasting 88040ms (88.04 seconds)

Experiment 'p_4_6_find_minimum_array' Results

June 3, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_4_6_find_minimum_array
Natural language statement: Write a method that finds the minimum value from a list of inputs into a method.
Method signature: p_4_6_find_minimum_array(inputs: array<int>) returns (min: int)

Ensures

- `forall i :: 0 <= i < inputs.Length ==> min <= inputs[i]`
- `exists i :: 0 <= i < inputs.Length && min == inputs[i]`

Requires

- `0 < inputs.Length`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method that finds the minimum value from a list of inputs into a method.

The signature should be:

p_4_6_find_minimum_array(inputs: array<int>) returns (min: int)

The method should respect the following contract:

requires 0 < inputs.Length, ensures forall i :: 0 <= i < inputs.Length ==> min <= inputs[
    ↪ i], ensures exists i :: 0 <= i < inputs.Length && min == inputs[i]

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DmfdclsZ1R64SFsLoazS5fBOeEFTR
Model: gpt-5.5-2026-04-23
Created at: 1780492172
Finish reason: stop
Usage: 332 tokens in, and 494 tokens out

```
{
  min := inputs[0];
  var idx := 1;
  while idx < inputs.Length
    invariant 1 <= idx <= inputs.Length
    invariant forall i :: 0 <= i < idx ==> min <= inputs[i]
    invariant exists i :: 0 <= i < idx && min == inputs[i]
  {
    if inputs[idx] < min {
      min := inputs[idx];
    }
    idx := idx + 1;
  }
}
```

Final Program

```
method p_4_6_find_minimum_array(inputs: array<int>) returns (min: int)
```

```
      requires 0 < inputs.Length
      ensures forall i :: 0 <= i < inputs.Length ==> min <= inputs[i]
      ensures exists i :: 0 <= i < inputs.Length && min == inputs[i]
{
  min := inputs[0];
  var idx := 1;
  while idx < inputs.Length
    invariant 1 <= idx <= inputs.Length
    invariant forall i :: 0 <= i < idx ==> min <= inputs[i]
    invariant exists i :: 0 <= i < idx && min == inputs[i]
  {
    if inputs[idx] < min {
      min := inputs[idx];
    }
    idx := idx + 1;
  }
}
```

Total Token Usage

Input tokens: 332
Output tokens: 494
Reasoning tokens: 369
Sum of 'total tokens': 826

Experiment Timings

Overall Experiment started at 1780492171920, ended at 1780492187740, lasting 15820ms (15.82 seconds)
Iteration #1 started at 1780492171920, ended at 1780492187740, lasting 15820ms (15.82 seconds)

Experiment 'p_4_6_find_minimum_seq' Results

June 3, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_4_6_find_minimum_seq
Natural language statement: Write a method that finds the minimum value from a list of inputs into a method.
Method signature: p_4_6_find_minimum_seq(inputs: seq<int>) returns (min: int)

Ensures

- `forall i :: 0 <= i < |inputs| ==> min <= inputs[i]`
- `exists i :: 0 <= i < |inputs| && min == inputs[i]`

Requires

- `0 < |inputs|`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method that finds the minimum value from a list of inputs into a method.

The signature should be:

p_4_6_find_minimum_seq(inputs: seq<int>) returns (min: int)

The method should respect the following contract:

requires 0 < |inputs|, ensures forall i :: 0 <= i < |inputs| ==> min <= inputs[i],
    ↪ ensures exists i :: 0 <= i < |inputs| && min == inputs[i]

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DmfdHbRhGD2K0OC2aTN4E88Y02saQ
Model: gpt-5.5-2026-04-23
Created at: 1780492151
Finish reason: stop
Usage: 338 tokens in, and 693 tokens out

```
{
  min := inputs[0];
  var i := 1;
  var k := 0;

  while i < |inputs|
    invariant 1 <= i <= |inputs|
    invariant 0 <= k < i
    invariant min == inputs[k]
    invariant forall j :: 0 <= j < i ==> min <= inputs[j]
  {
    if inputs[i] < min {
      min := inputs[i];
      k := i;
    }
    i := i + 1;
  }

  assert 0 <= k < |inputs| && min == inputs[k];
  assert exists j :: 0 <= j < |inputs| && min == inputs[j];
```

```
}
```

Final Program

```
method p_4_6_find_minimum_seq(inputs: seq<int>) returns (min: int)
       requires 0 < |inputs|
       ensures forall i :: 0 <= i < |inputs| ==> min <= inputs[i]
       ensures exists i :: 0 <= i < |inputs| && min == inputs[i]
{
  min := inputs[0];
  var i := 1;
  var k := 0;

  while i < |inputs|
    invariant 1 <= i <= |inputs|
    invariant 0 <= k < i
    invariant min == inputs[k]
    invariant forall j :: 0 <= j < i ==> min <= inputs[j]
  {
    if inputs[i] < min {
      min := inputs[i];
      k := i;
    }
    i := i + 1;
  }

  assert 0 <= k < |inputs| && min == inputs[k];
  assert exists j :: 0 <= j < |inputs| && min == inputs[j];
}
```

Total Token Usage

Input tokens: 338
Output tokens: 693
Reasoning tokens: 512
Sum of 'total tokens': 1031

Experiment Timings

Overall Experiment started at 1780492150588, ended at 1780492171091, lasting 20503ms (20.50 seconds)
Iteration #1 started at 1780492150588, ended at 1780492171091, lasting 20503ms (20.50 seconds)

Experiment 'p_4_8_seperate_lines' Results

June 3, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_4_8_seperate_lines
Natural language statement: Write a method that reads a word and returns each character of the word on a separate line.
Method signature: p_4_8_seperate_lines(s: string) returns (chars: string)

Ensures

- `|chars| == 2 * |s|`
- `forall i :: 0 <= i < |s| ==> i % 2 == 1 ==> chars[i] == '\n'`
- `forall i :: 0 <= i < |s| ==> i % 2 == 0 ==> chars[i] == s[i / 2]`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method that reads a word and returns each character of the word on a separate
    ↪ line.

The signature should be:

p_4_8_seperate_lines(s: string) returns (chars: string)

The method should respect the following contract:

ensures |chars| == 2 * |s|, ensures forall i :: 0 <= i < |s| ==> i % 2 == 1 ==> chars[i]
    ↪ == '\n', ensures forall i :: 0 <= i < |s| ==> i % 2 == 0 ==> chars[i] == s[i / 2]

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-Dmfe4pOhcvoskJsqQ5OxGkNsEwIfr
Model: gpt-5.5-2026-04-23
Created at: 1780492200
Finish reason: stop
Usage: 394 tokens in, and 2132 tokens out

```
{
  chars := seq(2 * |s|, i requires 0 <= i < 2 * |s| => if i % 2 == 0 then s[i / 2] else
      ↪ '\n');
}
```

Final Program

```
method p_4_8_seperate_lines(s: string) returns (chars: string)
       ensures |chars| == 2 * |s|
       ensures forall i :: 0 <= i < |s| ==> i % 2 == 1 ==> chars[i] == '\n'
       ensures forall i :: 0 <= i < |s| ==> i % 2 == 0 ==> chars[i] == s[i / 2]
{
  chars := seq(2 * |s|, i requires 0 <= i < 2 * |s| => if i % 2 == 0 then s[i / 2] else
      ↪ '\n');
}
```

Total Token Usage

Input tokens: 394
Output tokens: 2132
Reasoning tokens: 2047
Sum of 'total tokens': 2526

Experiment Timings

Overall Experiment started at 1780492199763, ended at 1780492243934, lasting 44171ms (44.17 seconds)
Iteration #1 started at 1780492199763, ended at 1780492243934, lasting 44171ms (44.17 seconds)

Experiment 'p_4_9_reverse_string' Results

June 3, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_4_9_reverse_string
Natural language statement: Write a method that reads a word and returns the word in reverse.
Method signature: p_4_9_reverse_string(s: string) returns (reversed: string)

Ensures

- `|reversed| == |s|`
- `forall i :: 0 <= i < |s| ==> reversed[i] == s[|s| - 1 - i]`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method that reads a word and returns the word in reverse.

The signature should be:

p_4_9_reverse_string(s: string) returns (reversed: string)

The method should respect the following contract:

ensures |reversed| == |s|, ensures forall i :: 0 <= i < |s| ==> reversed[i] == s[|s| - 1
    ↪ - i]

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-Dmfemd5ur1otB0QdwOSGV0lfnRWhl
Model: gpt-5.5-2026-04-23
Created at: 1780492244
Finish reason: stop
Usage: 318 tokens in, and 6111 tokens out

```
{
  reversed := "";
  var i := 0;
  while i < |s|
    invariant 0 <= i <= |s|
    invariant |reversed| == i
    invariant forall j :: 0 <= j < i ==> reversed[j] == s[|s| - 1 - j]
  {
    ghost var oldI := i;
    ghost var oldReversed := reversed;
    assert oldI < |s|;
    assert |oldReversed| == oldI;
    assert forall j :: 0 <= j < oldI ==> oldReversed[j] == s[|s| - 1 - j];

    reversed := reversed + [s[|s| - 1 - i]];
    assert reversed == oldReversed + [s[|s| - 1 - oldI]];
    i := i + 1;
    assert i == oldI + 1;
    assert |reversed| == i;
```

```
    forall j | 0 <= j < i
      ensures reversed[j] == s[|s| - 1 - j]
    {
      if j < oldI {
        assert oldReversed[j] == s[|s| - 1 - j];
        assert reversed[j] == oldReversed[j];
      } else {
        assert j == oldI;
        assert reversed[j] == s[|s| - 1 - oldI];
        assert |s| - 1 - oldI == |s| - 1 - j;
      }
    }
  }
  assert i == |s|;
  assert forall j :: 0 <= j < |s| ==> reversed[j] == s[|s| - 1 - j];
}
```

Final Program

```
method p_4_9_reverse_string(s: string) returns (reversed: string)
       ensures |reversed| == |s|
       ensures forall i :: 0 <= i < |s| ==> reversed[i] == s[|s| - 1 - i]
{
  reversed := "";
  var i := 0;
  while i < |s|
    invariant 0 <= i <= |s|
    invariant |reversed| == i
    invariant forall j :: 0 <= j < i ==> reversed[j] == s[|s| - 1 - j]
  {
    ghost var oldI := i;
    ghost var oldReversed := reversed;
    assert oldI < |s|;
    assert |oldReversed| == oldI;
    assert forall j :: 0 <= j < oldI ==> oldReversed[j] == s[|s| - 1 - j];

    reversed := reversed + [s[|s| - 1 - i]];
    assert reversed == oldReversed + [s[|s| - 1 - oldI]];
    i := i + 1;
    assert i == oldI + 1;
    assert |reversed| == i;

    forall j | 0 <= j < i
      ensures reversed[j] == s[|s| - 1 - j]
    {
      if j < oldI {
        assert oldReversed[j] == s[|s| - 1 - j];
        assert reversed[j] == oldReversed[j];
      } else {
        assert j == oldI;
```

```
      assert reversed[j] == s[|s| - 1 - oldI];
      assert |s| - 1 - oldI == |s| - 1 - j;
    }
   }
  }
  assert i == |s|;
  assert forall j :: 0 <= j < |s| ==> reversed[j] == s[|s| - 1 - j];
}
```

Total Token Usage

Input tokens: 318
Output tokens: 6111
Reasoning tokens: 5631
Sum of 'total tokens': 6429

Experiment Timings

Overall Experiment started at 1780492244669, ended at 1780492356131, lasting 111462ms (111.46 seconds)
Iteration #1 started at 1780492244672, ended at 1780492356131, lasting 111459ms (111.46 seconds)